

FREECODECAMP x EXAMPRO | COURSE STUDY GUIDE

GenAI Essentials

The Complete Study Guide

From machine-learning basics to transformers, prompting, RAG, local models, GPUs and agents: built from the full 22h 53m course (video parts 0-4), its transcript and on-screen slides.

Prepared for Priyanshu Jindal | Instructor in source course: Andrew Brown (with Rola Dali)

Contents

Course Orientation, Exam and the GenAI Roadmap	8
1.1 What is the GenAI Essentials certification?	8
1.2 Who is it for?	8
1.3 How long to study and how to pass	9
1.4 Exam facts (from the "Exam Guide" slides)	9
1.5 The GenAI Roadmap (maturity model)	10
1.6 Free learning resources mentioned	11
AI, Machine Learning, Deep Learning and Generative AI	12
2.1 The four nested terms	12
2.2 AI versus generative AI	13
2.3 Modalities of GenAI	13
2.4 Jupyter: the notebook environment used everywhere	13
2.5 Natural Language Processing (NLP)	14
2.6 Why this matters for the rest of the course	15
Classical Machine Learning	16
3.1 The three core tasks: regression, classification, clustering	16
3.2 Why the algorithm names matter	17
3.3 Types of machine learning (the overview slide)	17
3.4 Division of machine learning	18
3.5 Supervised vs unsupervised vs reinforcement learning	18
3.6 A tiny supervised example (for intuition)	19
3.7 Where classical ML still wins	19
3.8 Self-check	20
Neural Networks, Perceptrons and Activation Functions	21
4.1 Neural networks and deep learning	21
4.2 The perceptron	22
4.3 Activation functions	22
4.4 How a network learns: the loop in plain language	24
4.5 Self-check	24
BERT, Encoder Models and Sentence Transformers	25
5.1 What BERT is	25

5.2 Encoder-only models: what they are good at	26
5.3 Hands-on: running BERT with Hugging Face transformers	26
5.4 Sentence Transformers (SBERT)	27
5.5 BERT vs GPT at a glance	28
5.6 Self-check	28
Core ML Terminology and Responsible AI	29
6.1 Algorithm and function	29
6.2 What is an ML model?	29
6.3 What is a feature?	30
6.4 What is inference?	30
6.5 Parameters and hyperparameters	30
6.6 Responsible AI	31
6.7 Glossary refresher for this chapter	32
Foundation Models, LLMs and the Transformer	33
7.1 Foundation models	33
7.2 Large language models (LLMs)	33
7.3 The Transformer architecture	34
7.4 Tokenization	35
7.5 Tokens and capacity	35
7.6 Vectors and embeddings	36
7.7 Positional encoding	36
7.8 Attention	36
7.9 Parameters and model size	37
7.10 Self-check	37
Fine-Tuning, Pruning and Model Adaptation	38
8.1 Why adapt a model?	38
8.2 What is fine-tuning?	38
8.3 Types of fine-tuning	39
8.4 Pruning	39
8.5 Optimisation tricks met later in the labs	40
8.6 Fine-tuning vs RAG vs prompting	40
8.7 Self-check	40
Data Foundations for Machine Learning	41
9.1 Why a data chapter in a GenAI course?	41

9.2 Data labeling and ground truth	41
9.3 Data mining	41
9.4 Knowledge mining	42
9.5 Data wrangling	42
9.6 Data modelling and data analytics	43
9.7 Data roles	43
9.8 Training, validation and test data sets	44
9.9 Corpus and corpus linguistics	44
9.10 Data types	44
9.11 Self-check	45
Choosing Models: Leaderboards, Benchmarks and AI Assistants	46
10.1 Why compare models?	46
10.2 Reading a leaderboard such as Artificial Analysis	46
10.3 Benchmarks: what they measure	47
10.4 The Chinchilla paper	47
10.5 Choosing a model: a practical procedure	47
10.6 The AI-powered assistants tour	48
10.7 Self-check	48
Development Environments for GenAI Work	49
11.1 The big picture	49
11.2 Google Colab basics	49
11.3 SageMaker Studio Lab and Gradio	50
11.4 Lightning AI, GitPod and Deepnote	50
11.5 Local Python with Miniconda	50
11.6 JupyterLab server	51
11.7 Choosing quickly	51
Prompt Engineering	53
12.1 What is prompt engineering?	53
12.2 Zero-shot prompting	53
12.3 Few-shot prompting (in-context learning)	53
12.4 Prompt chaining	54
12.5 Chain of thought (CoT)	54
12.6 Tree of thought (ToT)	54
12.7 CO-STAR framework	55
12.8 ReAct prompting (reason + act)	55

12.9 Choosing a technique	56
12.10 Self-check	57
Calling Model APIs and Cloud AI Platforms	58
13.1 Two ways to use a model	58
13.2 The universal pattern	58
13.3 OpenAI	59
13.4 Anthropic and the Workbench	59
13.5 Cohere	59
13.6 Amazon Bedrock	59
13.7 Google Gemini API and Vertex AI	60
13.8 Azure AI Foundry (Azure OpenAI)	60
13.9 Playgrounds and workbenches	60
13.10 Cost and hygiene checklist	61
13.11 Self-check	61
Hugging Face: Models, Pipelines, Spaces and Datasets	62
14.1 What is Hugging Face?	62
14.2 Running a model on a GPU studio	62
14.3 The pipeline abstraction	63
14.4 Hugging Face Spaces	63
14.5 Datasets and a small training run	64
14.6 Putting it together: fine-tuning workflow on HF	64
14.7 Exam-style takeaways	64
14.8 Self-check	65
Running Models Locally, Frameworks and Structured Output	66
15.1 Why run a model locally?	66
15.2 Ollama	66
15.3 OPEA and Intel	67
15.4 Llamafire	67
15.5 llama.cpp and GGUF	67
15.6 LangChain and LlamaIndex	68
15.7 Context (prompt) caching	68
15.8 Docker on Ubuntu: lessons from the lab	68
15.9 Structured output (forcing JSON)	69
15.10 Self-check	70

AI Coding Assistants	71
16.1 The task: a "Linktree clone"	71
16.2 GitHub Copilot	71
16.3 Windsurf and Cascade	71
16.4 Cursor	72
16.5 Amazon Q Developer and Gemini	72
16.6 Comparison from the lab	72
16.7 Practical guidance	72
16.8 Self-check	73
Building GenAI Apps: UI Generators, Gradio, Streamlit and Backends	74
17.1 From model to product	74
17.2 Prompt-to-UI generators (v0, Lovable)	74
17.3 Gradio	74
17.4 Streamlit	75
17.5 API keys for the app	76
17.6 Supabase and login	76
17.7 What to remember	76
17.8 Self-check	76
Hardware, ML Frameworks and Serving Platforms	78
18.1 Why hardware matters	78
18.2 CPU, GPU, TPU and accelerators	78
18.3 Frameworks: PyTorch, TensorFlow, Keras	79
18.4 Serving stacks and Intel's OPEA	79
18.5 Gaudi, Tiber Developer Cloud and RunPod	80
18.6 Groq	80
18.7 Cloudflare Workers AI	80
18.8 Choosing where to run	81
18.9 Self-check	81
Model Size, Memory and Optimisation	82
19.1 What does "size" mean?	82
19.2 Memory arithmetic	82
19.3 Number formats	83
19.4 Quantization	83
19.5 Knowledge distillation	84
19.6 Flash attention	84

19.7 Fine-tuning example on Lightning AI	84
19.8 Optimisation toolkit	84
19.9 Self-check	85
RAG and Vector Databases	86
20.1 Retrieval-augmented generation	86
20.2 Vector databases and stores	86
20.3 Lab: Pinecone	87
20.4 Lab: Elasticsearch	87
20.5 Lab: Amazon OpenSearch and Bedrock knowledge bases	87
20.6 Lab: Postgres as a vector store	88
20.7 Adding web data: scraping and search	88
20.8 Chunking, embedding and retrieval choices	88
20.9 RAG vs fine-tuning (recap)	89
20.10 Self-check	89
Agents: OpenHands and CrewAI	90
21.1 What is an agent?	90
21.2 OpenHands (formerly OpenDevin)	90
21.3 CrewAI	91
21.4 A reinforcement-learning demo	91
21.5 Design tips for agents	91
21.6 The end of the course	92
21.7 Self-check	92
Study Aids: Cheat Sheet, Glossary and Practice Questions	93
S.1 One-page cheat sheet	93
S.2 Glossary	94
S.3 Practice questions	95
S.4 Answers	96
S.5 Course map and where each topic lives	97
S.6 How to study this guide	97

Course Orientation, Exam and the GenAI Roadmap

Source: video part 0, 0:00-0:53 (transcript + slide OCR). Instructor Andrew Brown, with guest Rola Dali for the roadmap discussion.

1.1 What is the GenAI Essentials certification?

The course opens by describing itself as a **practical generative-AI certification**. It teaches the fundamental concepts of machine learning (ML) and AI, the different *modalities* of generative AI (text, image, audio, video, 3D), and puts a strong focus on **large language models (LLMs)** and on **working with GenAI workloads programmatically**, both with cloud services and with local LLMs. The approach is deliberately **cloud-vendor agnostic**.

Andrew explains why he built it: other GenAI certifications exist (AWS, Azure and Nvidia each have one) and are decent, but they are very tied to a single vendor and leave large gaps. They also assume knowledge you may not have. The aim of GenAI Essentials is to give you *round and broad* knowledge so that you can succeed in any area of GenAI regardless of the technical choices your employer or project makes.

Item	Detail from the course
Course code	EXP-GENAI-001 (exam code used on the ExamPro slides)
Status	Described as a "beta"-like first version; a v002 with more content is hoped for and gaps will be fixed by minor updates
Follow-up	The Free GenAI Bootcamp , a project-based bootcamp where you build real GenAI workloads
Style	Lecture-heavy plus a very large hands-on lab section

1.2 Who is it for?

The slides list two main audiences:

- **Learners preparing for the Free GenAI Bootcamp.** The bootcamp "hits the ground running", so it expects you to be able to code and to have general familiarity with the topics. This course supplies that prerequisite knowledge.
- **People who need broad, practical knowledge of GenAI solutions** so they have *technical flexibility* to move in any direction. Andrew frames the course as a CTO-style exploration: *Should I use cloud services or local LLMs? Should I fine-tune or use RAG? How much does it cost? How do I secure it?* The course reports back on all of those questions.

The focus is on **implementation**: delivering GenAI workloads that are secure and within budget. Because Andrew has a developer background, you will see a lot of hands-on work. His advice if the labs are beyond you: just watch and absorb. The goal is exposure to the most knowledge in the least time.

KEY IDEA

The course has two halves. About the first 4 hours are **slide lectures** (concepts, terminology, architecture). The remaining ~19 hours are **follow-along labs** (notebooks, APIs, local models, coding assistants, RAG, GPUs, agents). This study guide follows the same order.

1.3 How long to study and how to pass

The slides give a study-time range of **15 to 30 hours**: about 15 if you are experienced (for example you already hold another GenAI certificate), about 30 if you are a beginner. The recommended pace is **1 to 2 hours a day for 15 days**. Even AWS AI Practitioner or Azure AI Engineer holders are told they will be surprised how much of the material was out of scope in those exams.

The exam is described as not difficult: it exists to *evaluate* your knowledge, so the course is lecture-and-lab-heavy rather than practice-exam-heavy. To pass:

1. Watch the lectures.
2. Do the hands-on labs where you can. You will not always have an account for a given cloud, the right hardware (Windows, Mac or Linux machine) or enough compute. That is fine: watch instead. Free options are used wherever possible.
3. Take the free practice exam (one attempt) on ExamPro; paying is only needed for the actual certification exam and badge.

1.4 Exam facts (from the "Exam Guide" slides)

Topic	Detail
Passing score	Above 750 points , about 75% (aim above it, because question counts do not always split evenly)
Questions	65 scored questions, no unscored questions, no penalty for wrong answers
Margin for error	About 16 scored questions can be wrong
Formats	Multiple choice, multiple answer, and case studies (which can be worth more than one point)
Duration	2 hours exam time (about 130 minutes); seat time 160 minutes (instructions, proctor room check, NDA, feedback)
Where	Online, from home, delivered through ExamPro's own proctoring system (TeacherSeat Anchor)
Proctoring	May be recorded and audited afterwards instead of a live human proctor
Validity	No expiry : there is no forced re-certification, but you may want to retake if a major new version appears

EXAM TIP

Remember the numbers 65 questions, 75% (750 points), 2 hours, and that the exam has *no unscored questions*. The seat time (160 min) is longer than the exam time (130 min) because it includes proctoring set-up and the NDA.

1.5 The GenAI Roadmap (maturity model)

Andrew and Rola Dali walk through a **GenAI roadmap** that started as a very detailed flowchart produced by Rola and colleagues. Andrew reorganised it into a **maturity model**, a way to step through the field and see where you are in your journey. It is *not* exhaustive, and not every item is in the course because of the time limit, but it shows the scope of what a GenAI practitioner should learn.

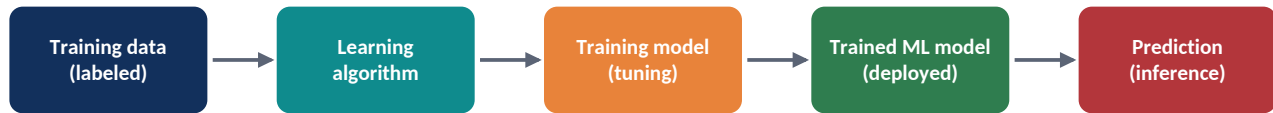


Figure 1.1: The basic ML flow that the whole roadmap builds on: labeled data goes into a learning algorithm, producing a trained model that makes predictions.

Stage by stage

The discussion covers these layers, roughly from foundations upward:

- **ML/AI foundations.** Rola's view: if you only want to set up one system you might skip ML basics, but if you want to *stay in the field* you need the bigger picture. Sooner or later you must learn it, and vectors, embeddings and the transformer architecture look intimidating but can be broken into small parts.
- **Modalities.** The main modalities are **text, image, video, audio and 3D**, plus niche ones such as protein folding and other health-science data. Video is a series of images and 3D files are vertices in text files, so the categories overlap. Multimodal models (for example text-to-image) blur which modality "wins".
- **Responsible AI, ethics and bias.** Included early because there is overlap with the LLM basics.
- **LLM basics.** Tokenization, context window, prompts. Andrew's point: if you truly understand what a **context window** is and what it can do, you have a lot of power with LLMs alone.
- **Prompt engineering.** Techniques you may already be using unconsciously.
- **AI-powered assistants** (ChatGPT, Claude, Copilot, Cursor and similar). Rola's team relies heavily on Claude and Cursor. The course's purpose is to help you **go beyond AI-powered assistants**.
- **Workbenches and playgrounds.** Andrew marks this as "where things start to get serious". They resemble assistants but expose more knobs such as **top-k and temperature**. Once you learn one, the others feel similar.
- **Model types and licensing.** Definitions of *open source*, *open weight*, *closed*, *edge-optimised* models (below).
- **Dev tools and workflows.** The GenAI life cycle and LLMOps (analogous to the software development life cycle and DevOps), Hugging Face ("the GitHub of AI/ML"), Ollama, and interoperability formats such as ONNX.
- **Serving and optimisation.** Inference engines (each big platform has its own, such as SageMaker and Bedrock), distributed serving with tools like Ray, and model-optimisation tricks.
- **Data.** How much does data matter?

Open source vs open weight vs closed

Term	Meaning as explained in the video
Open source model	Weights, training data <i>and</i> code are open
Open weight model	The weights are downloadable and usable, but the training data and code are not public (Meta's Llama and Mistral are given as examples)
Closed model	Only available through the provider's API (OpenAI is the example)
Edge-optimised model	Smaller model designed to run on phones and edge devices instead of a server

KEY IDEA

Many models people call "open source" are really **open weight**. You can download the weights and run them, but you do not know exactly how they were trained.

Interoperability and ONNX

Rola explains **ONNX** as an *interoperability* format: a shared way of encoding models so the same systems can be used across different models and providers. She compares the AI space to healthcare interoperability, where every vendor does what it wants and it becomes a headache; every provider releases models with its own encoding and tokenizer and stores them on disk their own way, which makes common tooling harder.

Data still matters

Asked how important data is for LLM work, Rola answers that for *regular ML* data is the bread and butter: the model learns from data, so it must exist, be of good quality and be consistent. For LLMs, the burden shifts mostly to the model builders, but you still benefit from knowing **what mix of sources a model saw** (social media, literature, science): "like asking whether someone went to college or spent ten years on social media". Andrew jokes about the movie *Idiocracy* to make the point.

Edge and cost

The pair discuss **GenAI on the edge** (AI on phones) as a predicted big trend and stress understanding how many resources these models use, because it raises questions about sustainability and profitability of hosted services, and whether to choose a smaller model you can run yourself.

WATCH OUT

The roadmap is a **learning map**, not a checklist of exam topics. Do not try to memorise every box; use it to see what you have covered and what comes next.

1.6 Free learning resources mentioned

Andrew ends by noting that many free resources exist right now from AWS, Google Cloud, Hugging Face and small start-ups, and that the current period is "the best time to learn". A curated list of these lives on the GenAI bootcamp site under the freeCodeCamp section, including some of his own videos.

CHAPTER 2

AI, Machine Learning, Deep Learning and Generative AI

Source: video part 0, 0:54-1:07 and 3:00-3:07 (transcript + slides "What is AI?", "AI vs Generative AI", "Natural Language Processing").

2.1 The four nested terms

Andrew starts the lecture series by putting four terms side by side, because people constantly say "AI" when they mean ML or deep learning. People will usually understand you anyway, so it is not a big deal if you mix them up, but you should know the difference.

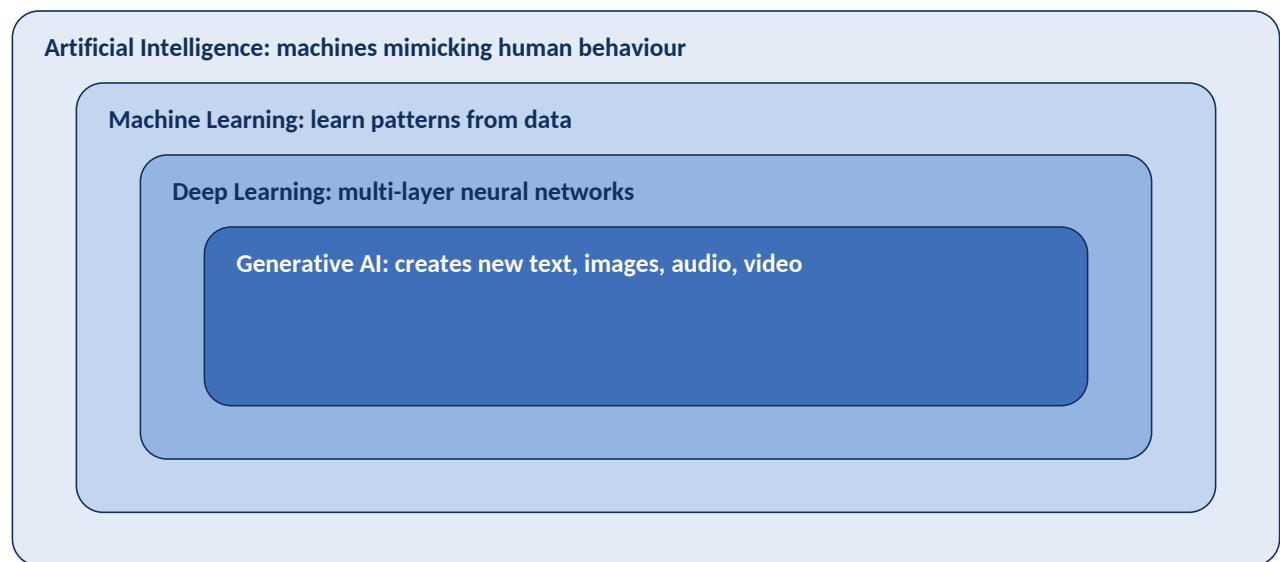


Figure 2.1: AI is the broadest term; machine learning, deep learning and generative AI are progressively narrower subsets.

Term	Definition given in the course	Typical examples
Artificial Intelligence (AI)	Machines that perform jobs that mimic human behaviour	Customer-service chatbots, recommendation systems, autonomous vehicles, medical diagnosis
Machine Learning (ML)	A subset of AI in which machines get better at a task without being explicitly programmed , by learning from data	Fraud detection, forecasting, spam filtering
Deep Learning (DL)	A subset of ML that uses multi-layered neural networks	The "nodes connected in layers" picture; image and speech recognition
Generative AI (GenAI)	A specialised subset of AI that generates new content such as text, images, video and audio	LLMs, image generators, music and speech synthesis

Andrew points out that a lot of people call neural networks "machine learning" or "AI", but that layered picture is specifically **deep learning**. GenAI is more of a marketing term. It does not fit neatly on the diagram (which is why it is drawn last), but because most GenAI techniques (LLMs, vision models) use neural networks, it technically **often relies on deep learning**.

KEY IDEA

GenAI is **not the same thing as LLMs**. An LLM is one modality (text) and one application of the broader idea of generating content.

2.2 AI versus generative AI

The slides define **generative AI** as a subset of AI focusing on **creating new content or data that is novel and realistic**. A GenAI system can *interpret or analyse* data but also *generate* new data itself. The content types include text, images, music, speech and other media. Advanced ML techniques used include:

- **GANs** (generative adversarial networks)
- **VAEs** (variational autoencoders)
- **Transformer models**, the architecture behind most current LLMs (see Chapter 7)

Side-by-side comparison

Aspect	Traditional AI	Generative AI
Functionality	Understanding and decision making	Creating new and original outputs
Data handling	Analyses existing data and makes decisions	Uses existing data to generate new, unseen outputs
Applications	Data analysis, automation, NLP, healthcare	Content creation, synthetic data, deepfakes, design
Nature	Analytical	Creative and innovative

Andrew notes that traditional AI can also generate something, but GenAI has the *added* benefit of generation as its core purpose.

Everyday examples of AI

Customer-service chatbots ("I hate them, but that is probably the number one use"), e-commerce recommendation systems (Amazon), autonomous vehicles and medical diagnostics. These are broad, cross-industry applications.

2.3 Modalities of GenAI

When the roadmap discussion introduced modalities, the list was **text, image, video, audio and 3D**. Multi-modal models work across more than one. Video-to-video or text-to-image cases blur the boundary, so treat modality as "the type of data going in and coming out".

2.4 Jupyter: the notebook environment used everywhere

Before ML topics, the course pauses to explain the tools you will keep meeting. **Jupyter** is the family of notebook technologies used for interactive ML work:

Tool	What it is
Jupyter Notebook	The classic browser-based notebook: cells of code and text with rich output

Tool	What it is
JupyterLab	The next-generation interface with tabs and side panels; it has largely replaced the classic notebook, although you still meet classic notebooks in some places
JupyterHub	A server that runs JupyterLab for multiple users : intended for classes, corporate data-science groups and research groups

Andrew adds that many vendors build on top of these. Amazon SageMaker Studio Classic is one example, extending JupyterLab (and Amazon later changed direction on extensions), so "notebook-like experiences" appear in many cloud consoles. Google Colab and Azure ML notebooks (Chapter 11) are the same idea again.

2.5 Natural Language Processing (NLP)

NLP is the branch of AI concerned with understanding and generating human language. The slide lists what NLP can do:

- Analyse and interpret text within documents, emails and messages
- Interpret or contextualise spoken text, for example **sentiment analysis**
- Synthesise speech (a voice assistant talking to you)
- Automatically translate spoken or written phrases between languages
- Interpret spoken or written commands and decide on an action
- Related: **information retrieval, question answering, topic modelling**

Andrew mentions **language understanding** as a specialised subset of NLP, more about older approaches. He then shows a simple flowchart of NLP terms whose only purpose is *exposure*: "not important to memorise, but so that when you see the terms later, you'll remember seeing them here".

NLP vocabulary from the slide

Group	Terms	Plain meaning
Text wrangling	Pre-processing, tokenization	Clean and split text before analysis
Structure and syntax	Part-of-speech tagging, chunking, dependency parsing, constituency parsing	Is a word a noun or adjective? Break text into chunks; find which word depends on which; build a "tree" of phrases
Processing functions	Named entity recognition (NER), n-grams, sentiment analysis, information extraction	Highlight important nouns or PII; sequences of n words; positive or negative tone; pull facts from text

EXAM TIP

You are not expected to derive these, but you should be able to *recognise* NER (find names, places, PII), sentiment analysis (positive/negative), and part-of-speech tagging (noun/verb/adjective).

BEYOND THE VIDEO

Modern LLMs perform many classic NLP tasks (NER, sentiment, translation, summarisation) with prompting alone, without a separate model per task. The older pipeline steps above are still useful vocabulary for reading papers and library documentation (spaCy, NLTK).

2.6 Why this matters for the rest of the course

Every later chapter builds on the nesting in Figure 2.1:

1. Classical ML (Chapter 3) supplies the vocabulary: features, labels, training, inference.
2. Deep learning (Chapter 4) supplies neural networks, weights and activation functions.
3. Transformers and LLMs (Chapters 5 and 7) supply tokens, embeddings and attention.
4. The labs (Chapters 11 onward) put all of it to work through APIs and local tools.

Classical Machine Learning

Source: video part 0, 1:07-1:27 (slides "Regression", "Classification", "Clustering", "Types of Machine Learning", "Division of Machine Learning", "Supervised vs Unsupervised vs Reinforcement", "Supervised/Unsupervised Learning Models").

3.1 The three core tasks: regression, classification, clustering

Before dividing machine learning into families, Andrew introduces three tasks that appear again and again. Two of them need **labeled** data, one does not.



Figure 3.1: Regression predicts a number (distance from the fitted line is the error); classification predicts a category; clustering groups unlabeled points.

Regression

Regression is the process of finding a function that predicts a **continuous value** (a number) such as a future price or forecast. You draw a line through your data points. The distance between a data point and the regression line is called the **error**: points close to the line are well predicted, points far away are errors.

The algorithm you choose to draw the line changes the prediction. Loss measures named on the slide:

Measure	Idea
Mean squared error (MSE)	Average of squared errors, so large mistakes are punished heavily
Root mean squared error (RMSE)	Square root of MSE, back in the original units
Mean absolute error (MAE)	Average size of the errors, without squaring

Classification

Classification is the process of finding a function to divide a **labeled** data set into classes or categories. You predict a *category* to apply to the input: *Will it rain next Saturday? Will it be hot or cold? Is this transaction fraud?* The output lies on one side of a decision boundary or the other.

Clustering

Clustering is the process of grouping **unlabeled** data based on similarities and differences. Andrew's example: a group of dots close together is judged to be Windows machines and another group is Mac computers, but nobody told the model those labels beforehand. The keyword to remember is **unlabeled**, which is what separates it from classification.

KEY IDEA

Regression = predict a **number**. Classification = predict a **label** (needs labeled data). Clustering = **discover groups** (no labels).

EXAM TIP

If a question says "labeled data, predict a category" the answer is classification; "unlabeled, find natural groupings" is clustering; "predict a future value" is regression.

3.2 Why the algorithm names matter

Andrew shows many algorithm names for each task and says that you will not memorise them now, but that when you do classification, regression or clustering "you're going to see these names because you'll have to choose which algorithm to use". The names are grouped below so that they are familiar when they appear later.

Task	Algorithms named in the course
Classification	Logistic regression, k-nearest neighbours (KNN) , support vector machines (SVM), kernel SVM, naive Bayes, decision trees, random forest
Regression	Linear regression and related algorithms (which you pick changes how the line is drawn)
Clustering	K-means , DBSCAN , K-modes
Association	Algorithms that find "if X then Y" relationships (for example: people who buy bread often buy butter)
Dimensionality reduction	PCA (principal component analysis), LDA (linear discriminant analysis), GDA (generalised discriminant analysis), SVD (singular value decomposition), latent Dirichlet allocation

3.3 Types of machine learning (the overview slide)

The "Types of Machine Learning" slide is described as an overview so that you meet the terms once. It goes beyond the three familiar families:

- **Self-supervised learning.** The model can create labels from the data itself. (Andrew flags that this is his understanding and it is covered later. It is the training idea behind LLMs predicting the next token.)
- **Multi-instance learning.** Examples of unlabeled data are "bagged" together and labeled as a group.
- **Statistical inference.** *Inductive* (from specific examples to a general rule), *deductive* (general rules to specific outcomes) and *transductive* (predict specific examples from a specific domain).
- **Learning techniques:**
 - **Multitask:** fit one model on one data set that addresses multiple related problems.

- **Active:** the model can query a human operator during learning.
- **Online:** use available data and update the model before each prediction is made.
- **Transfer:** reuse what a model learned on one task for another (this idea underlies fine-tuning in Chapter 8).
- **Ensemble:** combine several models (bagging, boosting, stacking).

3.4 Division of machine learning

The next slide organises the field into **classical machine learning** and **deep learning**, and the classical branch into *supervised*, *unsupervised* and *reinforcement* learning. Andrew emphasises that classical ML is **not obsolete**: the nearest-neighbour algorithm was invented in 1967 and many companies could solve business problems with it today. "Just because they're old does not mean that they're not good; it's a matter of organisations knowing how to adopt classical machine learning."

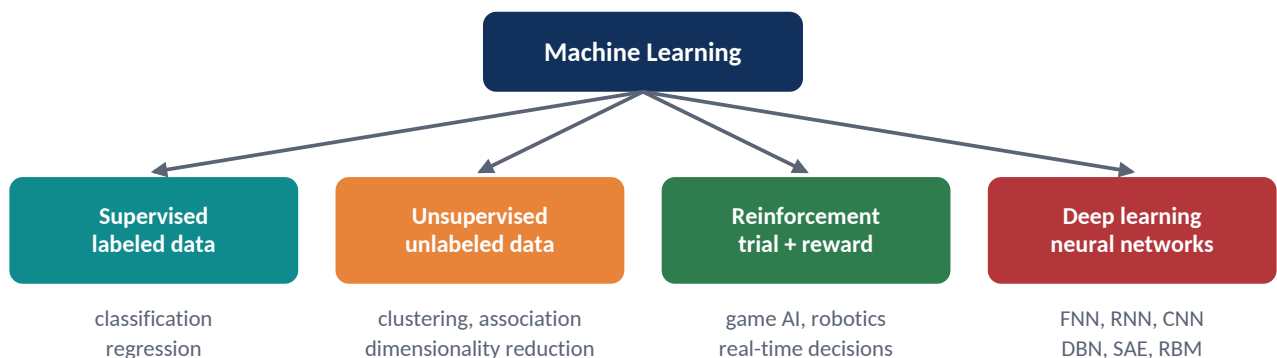


Figure 3.2: The division of machine learning used in the course.

3.5 Supervised vs unsupervised vs reinforcement learning

	Supervised	Unsupervised	Reinforcement
Data	Labeled ("the correct answers")	Unlabeled	No dataset: an environment with rewards
Learning style	"Show me how to do it and then I can do it on my own"	"I am an independent worker, I can figure this out"	Trial and error to find the right action
Driven by	The task (make a prediction)	The data (discover patterns, apply its own labels)	The decision (maximise reward)
Typical models	Classification, regression	Clustering, association, dimensionality reduction	Game-playing agents, robot navigation, autonomous driving
Trade-off	Usually more accurate but needs upfront labeling work	Less labeling, still needs human interpretation	Needs a simulator or a safe environment to fail in

Supervised learning models

Supervised learning needs **training data**: the inputs plus the correct answers. Two model types: **classification** and **regression**. A use-case for classification named in the course is **fraud detection**; for regression it is **market forecasting**. A worked classification example on the slides: given data on people's cholesterol, classify each person as **high risk** or **low risk**.

Unsupervised learning models

Needs no existing training data: it takes the unlabeled data, discovers patterns and applies its own labels. Three model types:

1. **Clustering**: group similar items (K-means, DBSCAN, K-modes).
2. **Association**: find relationships between variables. *If somebody buys bread, suggest butter, because past combinations suggest that people want it.*
3. **Dimensionality reduction**: reduce the amount of data while **retaining its integrity**, often used as a **pre-processing** stage before other models.

Reinforcement learning

Used when there is **no data**: the model works through trial and error and learns what the right thing to do is. Andrew's memorable example is an ML model playing **Mario or Sonic** again and again, failing until it can pass the level. Other uses: real-time decision making, game AI, learning tasks and **robot navigation** (autonomous driving).

Ensemble methods

When the quality of any one model or of the data is a problem, you can combine strategies across multiple models to get a better outcome: **bagging, boosting, stacking**. The word *boost* returns later in the course (for example gradient boosting).

EXAM TIP

A typical question gives a scenario and asks you to pick the learning type. Labeled data with a known answer means **supervised**; no labels and you want groups means **unsupervised**; an agent improving through rewards and penalties means **reinforcement**.

3.6 A tiny supervised example (for intuition)

BEYOND THE VIDEO

The course keeps the classical models conceptual. To make it concrete, here is what "training a classifier" looks like in Python with scikit-learn. The pattern (fit on labeled data, then predict) is exactly the ML flow from Figure 1.1.

```
from sklearn.neighbors import KNeighborsClassifier

X = [[170, 65], [180, 90], [160, 50], [185, 95]] # height, weight
y = ["small", "large", "small", "large"]         # labels

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)                                  # training
print(model.predict([[175, 80]]))                # inference
```

Notice the vocabulary that Chapter 6 formalises: **features** (**X**), **labels** (**y**), **training** (**fit**), **inference** (**predict**) and a **hyperparameter** (**n_neighbors**, the "K" in K-nearest neighbours that you set yourself).

3.7 Where classical ML still wins

- Small or tabular data sets (spreadsheets, sensor logs).

- Problems that need fast, cheap, explainable predictions.
- When you can measure error simply (MSE for prices, accuracy for classes).
- As a **pre-processing** step for GenAI work: clustering and dimensionality reduction are used to organise embeddings (Chapter 7).

3.8 Self-check

1. Which task predicts a continuous number, and what is the "error" in it?
2. Why is clustering called unsupervised?
3. Name two ensemble methods.
4. Which learning type would you choose for a robot learning to navigate a warehouse?
5. What is the purpose of dimensionality reduction in a pipeline?

Neural Networks, Perceptrons and Activation Functions

Source: video part 0, 1:25-1:27 and 2:38-2:53 (slides "Neural Networks and Deep Learning", "Introduction to Perceptrons", "Basic Perceptron Network", "Activation Functions" and one slide per function).

4.1 Neural networks and deep learning

Andrew's shortcut: **think of deep learning as neural networks**. It is the branch of ML used when the data is complicated, for example images, audio or language, and where hand-crafted features are hard to design. A network is a set of **nodes (neurons)** arranged in **layers** and connected by **weighted connections**.

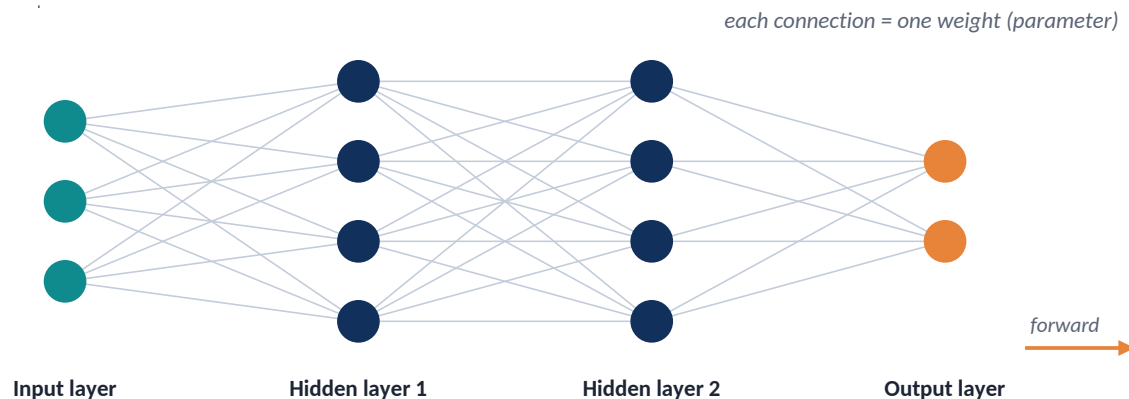


Figure 4.1: A small fully connected network. Data flows left to right; every line is a weight that training adjusts.

Vocabulary from the slides

Term	Meaning
Feed-forward neural network (FNN)	Connections between nodes do not form a cycle : data only moves forward, never backwards or sideways
Back-propagation	After a forward pass, the network moves backwards and adjusts the weights to improve the outcome on the next iteration. This is how a neural network learns
Loss function	Compares the <i>ground truth</i> (the correct answer) with the network's prediction to measure how wrong it is
Dense network	The number of nodes increases from layer to layer (adds information as it goes)
Sparse network	The number of nodes decreases from layer to layer

Deep-learning algorithm families

Like classical ML, deep learning has supervised and unsupervised sides. Andrew stresses it is "not important to really remember this, but I just wanted you to know":

- **Supervised** (labeled data): **FNN**, **RNN** (recurrent neural networks, sequence models), **CNN** (convolutional neural networks, typically images).

- **Unsupervised:** **DBN** (deep belief networks), **SAE** (stacked autoencoders), **RBM** (restricted Boltzmann machines).

4.2 The perceptron

The **perceptron** is the ancestor of every neural network. Andrew's point in spending time on such an old idea: *the concept is not new, we have simply scaled it*. We now have far more compute, and we no longer connect everything by hand. The neural-network idea comes from these connected layers, and "the fact that it's so old shows you that we've been doing ML longer than you think".

A basic perceptron network

- It has an **input layer** and an **output layer**; each layer contains a number of nodes.
- Nodes in adjacent layers have **established, weighted connections**.
- The input layer size matches the data (for example a thousand input nodes for a thousand values). The output layer matches the answer you want: if your classification is *yes or no*, you only need a **single output node**.
- Data moving between nodes is **multiplied by the weights**. A weight affects the *strength or weakness* of the value passed on.
- **Weights are modified during training** to produce a better outcome.

Hidden layers (extra layers between input and output) are not in the basic perceptron, but real networks add them; they let the network model complicated patterns.

4.3 Activation functions

Inside each node, **all the arriving inputs are summed** and then an **activation function** is triggered. The function acts as a **gate between nodes**: it decides whether the node is *active or inactive* and what value goes on to the next layer. Depending on the function, the output range can be 0 to 1 or -1 to 1, and so on. Andrew notes the list is not exhaustive and that in a more advanced ML course you would study each in detail.

Linear vs non-linear

- **Linear activation** (identity): passes data along unchanged. It **cannot do back-propagation**, **cannot stack layers**, so the model behaves as if it were a single linear layer and cannot handle complex non-linear data. Its range is unbounded and its derivative is 1. It is used for inputs (just passing data through).
- **Non-linear activations:** **can do back-propagation** and **can stack many layers**. Everything below is non-linear.

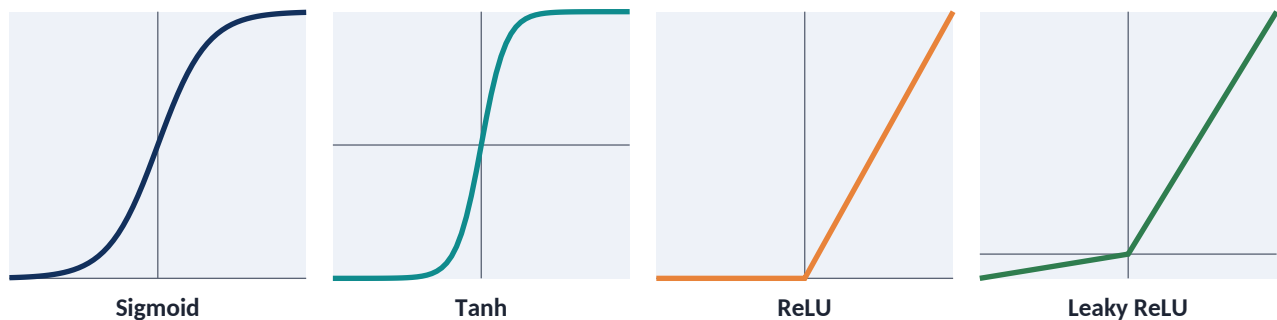


Figure 4.2: Shapes of four common activation functions. Sigmoid squashes to 0..1, tanh to -1..1, ReLU zeroes negatives, Leaky ReLU lets a small slope through.

The functions, one by one

Function	Behaviour	Notes from the course
Linear (identity)	Output = input; range unbounded	Cannot learn: no back-propagation, no stacking
Binary step	If above a threshold output 1 , else 0	Handles only binary classification (on/off, yes/no); one of the earliest used; very simplistic, rarely used today
Sigmoid	S-shaped logistic curve; output 0 to 1	Handles binary and multi-class problems; can stack layers; suffers the vanishing gradient ; "analog" so almost all neurons fire, making activation dense, slow and costly
Tanh	Like sigmoid but scaled; output -1 to 1 ; steeper gradient	Still has vanishing gradient; can outperform sigmoid depending on use case. (Andrew corrects himself: it is <i>tanh</i> , not "tan-H")
ReLU (rectified linear unit)	Negative inputs become 0 ; positive inputs pass unchanged; range 0 to infinity	The most commonly used ; triggers sparsely so it is cheaper and faster; can suffer the dying ReLU problem
Leaky ReLU	Small slope for negatives instead of zero	Counters dying ReLU : the negative axis "leaks" so nodes do not die
Parametric ReLU	Leaky ReLU with a configurable negative slope (0.01x in the slide)	A learnable/tunable version of the same idea
ReLU6	ReLU with an upper limit	Output is bounded to a maximum value
ELU (exponential linear unit)	Slopes towards -1 for negatives, linear for positives	Pushes mean activation towards zero, so faster learning and convergence ; avoids dying ReLU; saturates for large negative numbers
Swish	Smooth curve with a small dip below zero then linear	Proposed by the Google Brain team as a ReLU replacement; smooth and non-monotonic; family includes Mish, Hard Swish, Hard Mish
Maxout	Takes multiple inputs and returns the maximum	Generalises ReLU and Leaky ReLU; expensive because it doubles the parameters of each neuron
Softmax	Turns a vector of scores into probabilities that sum to 1	Used on the output layer for multi-class classification ; look for it near the outputs of the Transformer architecture diagram

Two problems to be able to explain

1. **Vanishing gradient.** In the flat parts of sigmoid and tanh the output barely responds to changes in input, so the gradient (the learning signal) shrinks towards zero. The network then "refuses to learn further or is painfully slow".
2. **Dying ReLU.** Because ReLU's gradient for negative inputs is exactly zero, a node whose inputs stay negative outputs zero forever and gets *stuck at zero*: nothing adjusts it any more, "the nodes essentially die". Leaky ReLU, parametric ReLU and ELU were invented to avoid that. Andrew notes "every time we look at one of these it is trying to solve a problem and be better".

KEY IDEA

Sigmoid and tanh fire almost every neuron (dense, costly). ReLU switches off negative neurons (sparse, cheap, fast). That trade-off explains why ReLU-family functions dominate hidden layers, while softmax dominates classification outputs.

EXAM TIP

Match the function to the property. Output between 0 and 1 for binary problems: **sigmoid**. Probabilities over many classes summing to 1: **softmax**. Negative values become zero: **ReLU**. Fixes dying ReLU with a small negative slope: **Leaky ReLU**. Output between -1 and 1: **tanh**. Google Brain replacement for ReLU: **Swish**.

4.4 How a network learns: the loop in plain language

BEYOND THE VIDEO

The video introduces the pieces; here they are put together as one loop, which is the same loop used to train LLMs at a vastly larger scale.

1. **Forward pass.** Inputs are multiplied by weights, summed, and passed through activation functions layer by layer until an output appears.
2. **Loss.** The loss function compares the output with the ground truth.
3. **Back-propagation.** The error is pushed backwards through the network to work out how each weight contributed.
4. **Update.** An optimiser (for example gradient descent) nudges each weight to reduce the loss. The size of the nudge is the **learning rate**, a hyperparameter (Chapter 6).
5. **Repeat** over many batches and epochs until the loss stops improving.

Andrew closes the section by reminding you that it is fine if it is not all clear yet: parameters, weights and layers are re-explained when the course reaches the Transformer, "so hopefully that is clear but we will cover it again".

4.5 Self-check

1. What is the difference between a feed-forward network and back-propagation?
2. Why can a network built only from linear activations not model complex data?
3. Which activation would you use on the output layer of a 10-class classifier?
4. Describe the dying ReLU problem and one activation function that mitigates it.
5. In a dense network, what happens to the number of nodes from layer to layer?

BERT, Encoder Models and Sentence Transformers

Source: video part 0, 1:27-2:38 (slide "BERT" plus a long hands-on notebook session and discussion with Rola Dali, and the slide "Sentence Transformers (SBERT)").

5.1 What BERT is

BERT stands for **Bidirectional Encoder Representations from Transformers**. It was released in **2018 by Google researchers**, and the models live on Hugging Face under the Google organisation. Andrew's easiest mental model: take the **Transformer architecture** from the paper *Attention Is All You Need*, keep **only the encoder**, and stack it on itself multiple times, and you get BERT.

Key properties from the slide:

- **Bidirectional:** it reads text **left to right and right to left**, so each word is understood in the context of the words on *both* sides. That makes it contextually aware of sentences and very good at NLP tasks. (Later in the discussion the pair note that "bidirectional" is somewhat of a misnomer, and *non-directional* might describe it better, because the whole sentence is processed at once.)
- **Encoder-only:** it turns text into rich numeric representations. It is *not* built to generate long text.
- **Pre-trained, then adapted:** in its raw pre-trained state you do not do much with it. You train it further for a specific task.

How BERT was pre-trained

Pre-training task	What happens
Masked language modelling (MLM)	Some tokens in a sentence are masked , and the model must fill in the blanks
Next-sentence prediction (NSP)	Given sentences A and B, predict whether B follows A

Tasks BERT is adapted for

Named-entity recognition (NER), question answering, sentence-pair tasks, summarisation, feature extraction, embeddings and more; the slide lists these under "BERT is pre-trained on the following tasks" and its downstream uses.

Sizes

BERT comes in many sizes. Andrew names **tiny** (about 4 million parameters), **base** (about 100 million) and **large** (a few hundred million), plus roughly two dozen additional variants. It was trained mainly on **English** text (the Toronto BookCorpus and English Wikipedia), though multilingual variants exist; he found a *BERT Japanese* variant while learning Japanese himself.

KEY IDEA

BERT = encoder-only. It is great when *input is a lot of text and output is less text*: a label, a sentiment, an entity, a vector. GPT-style models are **decoder-based** and generate text.

5.2 Encoder-only models: what they are good at

In the discussion, Rola Dali explains the practical picture:

- A language model takes natural language in and produces a **mathematical representation** out. Add a few neural layers on top and you can build different heads for different tasks.
- BERT-style models excel at **classification** and **sentiment analysis**, the most common uses she has seen. "Input is text, a lot of it, and output is less text: a word, a sentiment, a classification."
- Encoder-only models are good at **analysing text to reach a conclusion**, not at generating a sequence. Sequence-to-sequence systems (encoder + decoder) do generation.
- **Emergent tasks**: larger models, trained for one thing, turn out to do other things well. The bigger the model, the more emergent tasks it shows. This is why people were surprised by how much GPT models can do.

Parameters are fixed before training

A frequent confusion Andrew raises: does training more make the model *file* bigger? Rola clarifies that the **number of parameters is constant** even before training. For example, a 340-million-parameter model has 340 million parameters that are **initialised randomly**; training changes their *values*, not their count. The model makes a prediction, measures the error, goes back and adjusts (the loop in Chapter 4).

Names in the wild

Andrew asks about "distil-whisper", "Omni" and similar labels. Answer: there is **no naming convention**. These come from research labs and companies who name them as they like. **Distilled** usually signals *knowledge distillation*, a smaller model trained to imitate a larger one (Chapter 8).

5.3 Hands-on: running BERT with Hugging Face `transformers`

The lab part of the video walks through the Hugging Face model page for `bert-base-uncased` in a notebook. "Uncased" means the text is lower-cased before processing. The steps illustrate how *little code* a huge model needs, which Rola connects to the idea of **abstracting complexity away**: "a huge model with a few lines of code is impressive, but it also means there is a whole lot of complexity underneath."

```
!pip install -q transformers

from transformers import pipeline
classifier = pipeline("sentiment-analysis")
print(classifier("This restaurant is awesome, the food is delicious!"))
```

What the session demonstrated and hit along the way:

1. A model card usually contains example code you can copy, but you must read it: the Japanese BERT example needed extra libraries (`fugashi`, a wrapper for the **MeCab** Japanese morphological dictionary) before its **tokenizer** would run.
2. The output of a tokenizer includes **special tokens** such as a *start* marker and a *separator*, which BERT uses to identify sentence sections.
3. The `pipeline` helper downloads a default **fine-tuned** version (a BERT-family model already trained for sentiment analysis) and hides tokenization, model call and post-processing.

4. Trying the same workflow on Google's **Vertex AI** hit a **quota limit** on machine types; checking *IAM & Admin*, *Quotas* and the Compute Engine machine list was the troubleshooting step. It is a reminder that cloud GPUs and large machine types are often quota-gated for new accounts.

WATCH OUT

Following model-card code blindly does not always work. Missing dependencies, changed library versions and quota limits are normal. The course's advice: consult an expert or the docs when the output is unfamiliar.

5.4 Sentence Transformers (SBERT)

Sentence Transformers, also called **SBERT**, is a library and family of models **built on top of BERT** that creates a **single vector for a whole sentence** (or paragraph), instead of one vector per token.

Why it exists: if every *word* becomes a vector, the numbers add up quickly and comparing full sentences is awkward. Sentence Transformers solves BERT's limitation when you care about **comparing sentences**. The trade-off is a matter of use case, not quality: for a workload that is **token by token**, BERT may be better; for **sentence-level similarity**, use Sentence Transformers.

Uses listed on the slide: **embeddings**, **semantic search**, **retrieve-and-rerank**, **clustering**, **image search** and more. Andrew notes that some models expect you to use Sentence Transformers for their embeddings, so you should recognise when a model does.

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")
a = model.encode("The cat sits on the mat")
b = model.encode("A feline is resting on a rug")
print(util.cos_sim(a, b))      # close to 1 means similar meaning
```

In the lab he pulled **all-MiniLM-L6-v2** (a small, fast embedding model), encoded two sentences into **embeddings**, and compared them with a similarity measure. He also browsed Hugging Face's embedding-model listings, noticing law-specific and Chinese-language variants, and hit the classic notebook error **name 'SentenceTransformer' is not defined**, which simply meant the import cell had not been run.

KEY IDEA

Embedding models turn text into vectors so that meaning can be compared with maths. This is the foundation of semantic search and of RAG (Chapter 16).

5.5 BERT vs GPT at a glance

	BERT	GPT-style LLM
Transformer part	Encoder only	Decoder (generation)
Direction	Sees context on both sides	Predicts the next token from the left context
Pre-training	Masked language model + next-sentence prediction	Next-token prediction
Typical output	Label, entity, vector	Generated text
Typical use	Classification, NER, embeddings, search	Chat, writing, code, reasoning

EXAM TIP

"Which model architecture is best for classifying text or producing embeddings?" points to an **encoder-only** model such as BERT. "Generating long-form text" points to a **decoder-based** LLM.

5.6 Self-check

1. What do the letters in BERT stand for and which part of the transformer does it use?
2. Name BERT's two pre-training tasks.
3. Why does the parameter count stay the same before and after training?
4. When would you choose Sentence Transformers over plain BERT?
5. What does "uncased" mean in **bert-base-uncased**?

Core ML Terminology and Responsible AI

Source: video part 0, 2:53-3:03 (slides "What is an Algorithm and Function?", "What is an ML Model?", "What is a Feature?", "What is Inference?", "What are Parameters and Hyperparameters?", "Core dimensions of responsible AI").

6.1 Algorithm and function

An **algorithm** is a set of mathematical or computer instructions to perform a specific task. It answers "how do you do something?" An algorithm can be composed of several smaller algorithms.

Andrew's example is **K-nearest neighbours (KNN)**: "tell me who your closest neighbours are and we will infer that you can be considered the same class." KNN can use different **distance metrics**, for example Euclidean, Hamming, Minkowski and Manhattan. He adds a subtle point: KNN by itself is just a distance-based method; it *becomes* a machine-learning algorithm when it is applied to solve a machine-learning problem, such as supervised classification.

A **function** is a way of grouping algorithms together so you can call them to compute a result. As the slide jokes, "sounds like an ML model".

6.2 What is an ML model?

In general terms, a **model** is an informational representation of an object, person or system. Models can be:

- **Concrete**: they have a physical form (a design for a vehicle, a person posing for a painting).
- **Abstract**: expressed as behavioural patterns (mathematics, computer code, written words).

An **ML model** is a *function that takes in data and performs a machine-learning algorithm to produce a prediction*.

Two words are easily confused:

Term	Meaning
Training model	The model <i>while</i> it is learning to make correct predictions
Trained model (the ML model)	The result once it has been tuned and deployed to make predictions

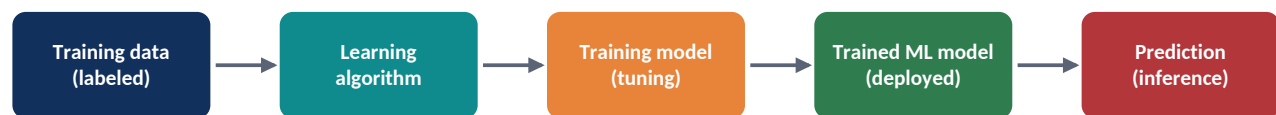


Figure 6.1: Labeled training data plus a learning algorithm produce a training model. After tuning it becomes the deployed, trained model, which turns unlabeled input into predictions.

Between training and deployment there is **hyperparameter tuning**: continuously tweaking the training settings until the model behaves as you want.

6.3 What is a feature?

A **feature** is a characteristic **extracted from an unstructured data set** and prepared so that an ML model can ingest it and infer a prediction. ML models generally only accept **numerical data**, so raw data is turned into a machine-readable format by **encoding**.

Feature engineering is the process of extracting features from data sources: take the raw data, **clean and transform** it into features, and feed those to the model. In the KNN example earlier, height and weight were the features.

6.4 What is inference?

Inference is the act of *requesting and getting a prediction*. You input data into a model that has been **deployed for production use**, and it outputs a prediction. The textbook meaning (moving from a premise to a logical consequence) is fine, but Andrew's easy-to-remember version is the request/response one.

His illustration: your raw data is a banana; you ask the deployed model "tell me what this is" and it replies "*a yellow banana*" with a **confidence score of 0.9**. That confidence score is a nice link back to the roadmap talk: predictive models generally return a probability or confidence, whereas a chat LLM gives you an answer without saying how sure it is.

KEY IDEA

Training is when the model learns (expensive, done once or occasionally). **Inference** is when the model is used (cheap per call but happens constantly, and is what you pay for on an API).

6.5 Parameters and hyperparameters

	Model parameter	Model hyperparameter
Where	Internal to the model	External to the model
Value	Learned from data during training; not set by hand	Set manually before training; cannot be estimated from data
Purpose	Used to make predictions	Used to estimate/learn the parameters
Examples	The weights on connections between nodes	Learning rate, epochs, batch size (and K in KNN)

Andrew's diagram: a value enters the model, flows through layers, and each connection between nodes carries a **weight**. Those connection weights *are* the parameters, "the internal state of the model". When the model trains, all those parameters must be set so that you get the result you want. This concept returns in Chapter 7, where the number of parameters becomes the standard measure of an LLM's size.

EXAM TIP

Parameters are learned, hyperparameters are set. If a question mentions learning rate, batch size or number of epochs, the answer is *hyperparameters*. If it mentions weights, the answer is *parameters*.

6.6 Responsible AI

Andrew presents responsible AI in the AWS-flavoured form used by the AWS AI Practitioner exam, and notes that Microsoft and others publish similar lists that are "more or less the same". The **core dimensions** are:

Dimension	Plain meaning
Fairness	Outcomes do not systematically disadvantage groups
Explainability	You can explain why the model made a decision
Privacy and security	Data is protected and used appropriately
Safety	The system does not cause harm
Controllability	Humans can steer or override the system
Veracity and robustness	Outputs are accurate and hold up under unusual input
Governance	Policies, oversight and accountability exist
Transparency	Users know how the system works and where it is used

The video points at AWS services connected to these ideas: **model evaluation on Amazon Bedrock**, **Amazon SageMaker Clarify** (for *explainable AI*, understanding what drives predictions and detecting bias), **guardrails** (filters on model input and output), and **model monitoring** (tracking degradation of a model over time). Each is covered where the course reaches that service, and the point for this course is the vocabulary.

Bias and data

The roadmap discussion had already tied ethics to data: a model reflects the sources it saw. If it learned mainly from social media, it will behave like a person who spent ten years on social media. Knowing the *mix of training data* (social media, literature, science) is a first step to reasoning about **bias**.

EXAM TIP

You may be given the list above and asked to identify an item, or given a scenario ("the team cannot explain why the model rejected a loan") and asked which dimension is missing (**explainability**).

BEYOND THE VIDEO

In practice, responsible-AI work includes **red-teaming** a model for harmful outputs, adding **guardrails** for topics and personal data, logging prompts and outputs for audit, and keeping a **human in the loop** for high-stakes decisions.

6.7 Glossary refresher for this chapter

Term	One-line definition
Algorithm	A set of instructions to do a task
Function	A callable grouping of algorithms
Model	A representation of a system; an ML model turns data into predictions
Feature	A numeric characteristic prepared for the model
Feature engineering	Cleaning and transforming raw data into features
Inference	Requesting and receiving a prediction from a deployed model
Parameter	A value learned in training (a weight)
Hyperparameter	A setting chosen before training (learning rate, epochs, batch size)

Foundation Models, LLMs and the Transformer

Source: video part 0, 3:05-3:25 (slides "What is a Foundational Model?", "What is a Large Language Model?", "Transformer Architecture", "Tokenization", "Tokens and Capacity", "What are Embeddings?", "Positional encoding", "Attention").

7.1 Foundation models

A **foundation model** (FM) is a **general-purpose model trained on vast amounts of data**: text, images, video, structured data, all kinds. It is called **pre-trained** because it can afterwards be **fine-tuned** for specific tasks. From one foundation model you can get prediction, classification, text generation and more, even if any single model is limited to certain things.

The terms **foundation model**, **base model** and **pre-trained model** are used almost interchangeably at this point in the pipeline. They do not mean *exactly* the same thing, but they describe the same artefact: the big general model before task-specific adaptation.

KEY IDEA

LLMs are a specialised subset of foundation models: foundation models that use the **Transformer architecture**. Not every foundation model is an LLM (for example image models), but every LLM is a foundation model.

7.2 Large language models (LLMs)

An **LLM** is a foundation model that implements the Transformer architecture. Natural language goes in; the model predicts the output word by word, and **as it produces each word it feeds it back in** and continues until it is done.

During **training** the model learns the *semantics or patterns of language*: grammar, word usage, sentence structure, style and tone. That is what makes it good at interpreting language and producing text that sounds right.

It is tempting to say "an LLM just predicts the next word". Andrew agrees that is the mechanical loop, but adds an honest caveat: **researchers do not fully know how LLMs generate their outputs**. With so many layers, the complexity makes it very hard to truly understand how a particular answer is reasoned; it *looks* word-for-word, but there is more to it.



Figure 7.1: The path of a prompt through an LLM. Each generated token is appended and the loop runs again.

7.3 The Transformer architecture

The Transformer was developed by researchers at **Google** and introduced in the paper ***Attention Is All You Need***. It is effective at NLP thanks to **multi-head attention** and **positional encoding**. Before it, the dominant sequence models were **RNNs** (recurrent neural networks) and **CNNs** (convolutional networks). RNNs could do something similar but **struggled to scale and to remember everything they were looking at**; the Transformer fixed that.

Andrew is candid about how much you need this: it is "good and nice to know", but once you work with LLMs day-to-day you mostly forget it and it does not drive your workflows. Its value is **confidence when reading white papers and architecture diagrams**.

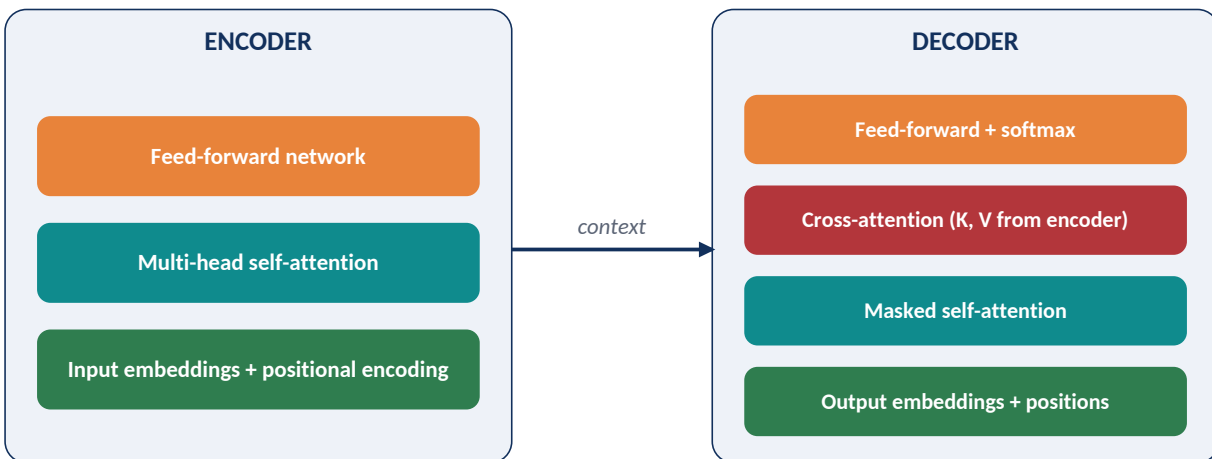


Figure 7.2: A simplified Transformer. The encoder reads and understands the input; the decoder generates the output one token at a time, using the encoder's context through cross-attention.

Encoder and decoder

- **Encoder:** "reads and understands the input text": like a smart system that picks up the meanings of words and how they are used in different contexts.
- **Decoder:** using what the encoder learned, it **generates new text**, like a skilled writer producing sentences that flow. Each new word is appended and the whole sequence is fed back in, looping until it runs out of ability or decides to stop.

Input must already be **embedded** before it enters the network (see 7.6). Models in practice come in three shapes:

Shape	Uses	Examples in the course
Encoder-only	Understand text: classify, embed, tag	BERT, Sentence Transformers
Decoder-only	Generate text	GPT-family and most chat LLMs
Encoder-decoder	Transform one sequence into another (translation, summarisation)	T5

7.4 Tokenization

Tokenization is breaking input (usually text) into smaller parts, then giving each part a **unique ID from the model's vocabulary**. LLMs require this: the input text must be converted into a sequence of tokens that match the model's **internal vocabulary**, created when the model was trained (all the text it consumed is broken into unique tokens and assigned values). Large models have vocabularies of roughly **30,000 to 100,000+ tokens**.

Different models use different tokenization algorithms:

Algorithm	Named as used by
Byte-pair encoding (BPE)	GPT-3
WordPiece	BERT
SentencePiece	Google T5 (Andrew also lists GPT-3.5 here as spoken)

You rarely see this when using Ollama or a managed API, because those handle it for you. A token is **not** a word: it may be a whole word, part of a word, or punctuation.

```
"Unbelievable results!" -> ["Un", "believ", "able", " results", "!"] -> [3118, 45021, 540, 2456, 0]
```

(illustrative split; the exact pieces and IDs depend on the model's tokenizer)

7.5 Tokens and capacity

The decoder **continuously feeds the sequence of tokens back in as input** to predict the next word: *The quick* becomes *The quick brown*, then *The quick brown fox*, and so on. Two resources grow with every token:

Resource	Effect of more tokens
Memory	Each token in the sequence needs memory; as the token count rises memory rises until it is exhausted and no more can be produced
Compute	The model performs more operations for each additional token , so longer sequences require more compute

Because of this, AI services that offer **Models-as-a-Service** often impose a limit on **combined input plus output** tokens. A huge prompt leaves less room to generate. This limit is the **context window**.

EXAM TIP

The **context window covers input and output together**. If a question says the response was cut off after a long prompt, the likely reason is the combined token limit.

To cut cost providers must make models more efficient. A practical trick for conversations that grow too long is to **summarise the conversation and feed the summary back in**, so the model keeps the gist without using all of the earlier tokens.

7.6 Vectors and embeddings

A **vector** is "an arrow with a length and a direction" (a mathematician would say more). It matters because vectors live in a **vector space**. A **vector space model** represents text documents or other data as vectors in a high-dimensional space. Distances between the points correlate with the relationships between the documents: vegetable documents cluster together, meat ones in another area, dairy in another.

Embeddings are the vectors of data ML models use to find relationships in data. Special ML models are built just to produce embeddings; Andrew's example is **Cohere**, whose *Command R* is a generative model while its *embed* models take an input and output embeddings to be stored in a **vector store**.

Different embedding algorithms capture different relationships: similarity in **spelling**, **word length**, or **contextual** relationships such as belonging to an industry. Real embedding models capture *many* relationships at once, which is why they are useful. Andrew describes embeddings as **external memory for performing a task**, and notes they can be **shared across models**, enabling **multimodal** patterns where models coordinate on a task.

```
"king"  -> [0.21, -0.44, 0.90, ...]  (hundreds or thousands of numbers)
"queen" -> [0.19, -0.40, 0.88, ...]  close to "king" in the vector space
"tofu"  -> [-0.60, 0.32, 0.05, ...]  far away
```

KEY IDEA

Similar meaning = nearby vectors. Semantic search, recommendations and RAG all work by measuring distance between embeddings.

7.7 Positional encoding

Transformers **do not process data sequentially**, so on their own they would lose word order. **Positional encoding** is the technique that **preserves the order of words**. RNNs handled order naturally because they read sequentially, but that made them hard to scale. In the architecture diagram positional encoding appears **right after the embeddings**:

1. Input text is **tokenized** into tokens.
2. Tokens are **embedded** into vectors.
3. Each gets a **positional vector**, so the model can keep track of order as the data is transformed.
4. Only then does it enter the Transformer layers.

7.8 Attention

Attention works out how important each word or token in a sequence is to the *other* words, by assigning **attention weights**. Three variants appear:

Type	What it does	Where
Self-attention	Computes weights within the same sequence : every element attends to all the others (the layer feeds back on itself)	Encoder; decoder (masked)

Type	What it does	Where
Cross-attention	Computes weights between two different sequences , so the decoder focuses on the encoder's input	Decoder, e.g. translation
Multi-head attention	Runs several attention heads in parallel , each focusing on a different aspect	Both, to capture varied dependencies at once

On the diagram, attention takes **Q, K and V** (query, key, value). Andrew's search analogy: type a *query*; it matches *keys*; you get back *values*. In **cross-attention**, K and V come from the **encoder** and Q from the **decoder**.

WATCH OUT

The course itself says you need not retain this detail; "it might be very hard to retain unless you are actually very invested in building these things". Aim to recognise the words and the encoder/decoder split.

7.9 Parameters and model size

Parameters are the **weights of the connections** between nodes. A connection may have one parameter (usually) or several, and because every node connects to every node in the next layer, the count grows very quickly. This is why LLM size is quoted in **billions of parameters** (Llama 8B, 70B and so on) and why memory needs (Chapter 19) depend on parameter count.

7.10 Self-check

1. Why are LLMs described as a subset of foundation models?
2. What does positional encoding fix, and where does it appear in the pipeline?
3. Which resources grow with every extra token?
4. What does cross-attention connect that self-attention does not?
5. Give one tokenization algorithm and a model that uses it.

Fine-Tuning, Pruning and Model Adaptation

Source: video part 0, 3:24-3:33 (slides "Fine Tuning LLMs", "Model layers") plus the optimisation lab section in video part 3 (19:50-20:30).

8.1 Why adapt a model?

If a model does not behave the way you want, you can do something to it to make it work better. To understand the ways of doing that, Andrew first reviews the internals. A model is made of **layers of nodes (neurons)** with **connections** between them. Connections often cross layers but can also loop within a layer (this is where **self-attention** comes from), and multiple sets of layers can be computed **in parallel**, which is **multi-head attention**.

Layers in well-known models

Model	Layers / size (as stated)
GPT-3	96 layers, 175 billion parameters. One of the smaller models you can still fine-tune, for instance in Azure AI Studio (older models like Babbage and Davinci are in this class)
BERT	12 layers, up to 24
GPT-2	12 to 48 layers (the same or more than BERT)
Google T5	12 encoder + 12 decoder layers, up to 24

Fine-tuning is essentially **tweaking which layers and connections you train**, and on what data.

8.2 What is fine-tuning?

Fine-tuning is **retraining a pre-trained model's weights (its parameters) on a smaller data set**. A model's *weights* are the saved output state of a trained model. The important clarification: **you are not creating new code or a brand-new model; you are producing a new set of weights**, a new saved state of the same model.



Figure 8.1: A foundation (base) model plus a smaller task-specific data set produces new weights. Those weights are the fine-tuned model.

Terminology check from the video: *foundation model*, *base model* and *pre-trained model* all refer to the starting point. After you fine-tune it, the result is the **fine-tuned model**.

Supervised fine-tuning (SFT)

In **supervised fine-tuning** the data set you provide is already **labeled**. Andrew's analogy: photos of animals with each animal named, so the model has "a cheat sheet". You explicitly tell the model what the data is. That contrasts with base-model training, which is often **unsupervised**: labeling an entire internet-scale corpus would be nearly impossible.

8.3 Types of fine-tuning

The video groups approaches by what you change. It notes the list is not exhaustive.

Change the data set

Approach	Idea	Example
Instruction fine-tuning	Give example pairs: "when a person says this, respond like that"	Teach the model to follow a house-style of answers
Domain-specific fine-tuning	Use a knowledge base of specific content so the model focuses on it	Make a generic model into a cloud-computing tutor, loaded with up-to-date cloud material

Change the training method

Approach	Idea	Trade-off
Full fine-tuning	All of the model's weights are updated, using the base weights as the starting point and re-running training	Most thorough but expensive
Parameter-efficient fine-tuning (PEFT)	Update only a small subset of parameters and freeze the rest	Saves money. LoRA is a well-known PEFT technique
Last-layer fine-tuning	Freeze every layer except the last ; train only that	Cheap; the video says it "apparently works really well"

Here "freeze" means saving the state at that point and telling training not to change it. The two dimensions combine: you can do *full* fine-tuning on a *domain-specific* data set, or PEFT on an *instruction* data set.

KEY IDEA

PEFT/LoRA = update few parameters; full fine-tuning = update all. The bigger the model, the more this matters for cost and hardware.

EXAM TIP

"Which technique lowers the cost of adapting a large model by training only a small set of parameters?" **PEFT (for example LoRA)**. "Retraining *all* weights" is **full fine-tuning**. Data with "input/desired output" pairs is **instruction** (supervised) fine-tuning.

8.4 Pruning

Pruning removes parameters (connections or neurons) to make a model **smaller and more efficient**. Less compute and faster inference come with trade-offs in quality. Two ways:

1. **Train-time pruning:** encourage the model to drop connections or neurons *during* training.
2. **Post-training pruning:** modify the model-weights file after training (Andrew's words: "mangling" the output file).

8.5 Optimisation tricks met later in the labs

In the GPU and inference lab section (Chapter 18), the same theme returns as a set of tricks to shrink huge models:

- **Quantization:** reduce numeric precision of weights, for example from 32-bit floats to 16, 8 or 4 bits. It compresses the model like lowering the quality of a JPEG: there is potential loss of quality, but a big gain in memory.
- **Knowledge distillation:** train a smaller "student" to imitate a bigger "teacher". The lecture mentions Nvidia's **Minitron**, which takes a large model down to smaller ones (for example from 15B to 8B to 4B parameters) using **distillation plus pruning**, aiming to perform nearly as well as the original (Nemotron).
- **Flash attention:** a memory-efficient, faster variant of attention optimised for GPUs, in variants 1, 2 and 3 (three is not always better than two).
- **Model merging** and other techniques mentioned in the roadmap talk.

BEYOND THE VIDEO

A quick way to remember the toolkit: **fine-tune** to change behaviour, **prune / quantize / distil** to change size, **RAG** (Chapter 20) to add *knowledge* without touching the weights.

8.6 Fine-tuning vs RAG vs prompting

Goal	Best first tool	Why
Change format, tone, or task behaviour	Prompt engineering (Chapter 12)	Free and instant
Give the model your private or up-to-date facts	RAG	Update documents, not weights
Teach a specialised style or domain reliably	Fine-tuning	Bakes it into weights
Run cheaply on small hardware	Quantize / distil / prune	Shrinks the model

Andrew's roadmap question (fine-tune or RAG? cloud or local?) is exactly this decision, and the rest of the course gives you the hands-on knowledge to answer it.

8.7 Self-check

1. What exactly does fine-tuning output: code, a model, or weights?
2. What does "freezing" layers mean and which technique uses it?
3. Give two ways to prune a model.
4. Which technique reduces numerical precision of weights?
5. For adding your company documents to an LLM's answers, would you fine-tune or use RAG first?

Data Foundations for Machine Learning

Source: video part 0, 3:32-3:48 (slides "What is Labeling?", "Data Mining", "Data Mining Methods", "Knowledge Mining", "What is Data Wrangling?", "What is Data Modeling?", "Data Analytics", "What is a Data Scientist?", "Data Role Comparisons", "Train vs Test vs Validation Dataset", "Corpus", "What is a Data Type?").

9.1 Why a data chapter in a GenAI course?

Andrew's stance: it is really important to understand data topics if you want to work with machine learning, because ML is "just complex algorithms predicting or forecasting things based on data". Even for LLMs, where the model builders handle most of the data burden, the vocabulary appears constantly in documentation, cloud consoles and exam questions.

9.2 Data labeling and ground truth

Data labeling is identifying raw data (images, text files, videos) and adding one or more meaningful, informative **labels** to give it context so an ML model can learn from it in **supervised learning**. Labeling is a **prerequisite to produce training data**, and is generally done by **humans**. An example on the slide: Amazon Rekognition drawing bounding boxes or classifying images into categories.

- **Supervised learning** requires human-labeled data.
- **Unsupervised learning** produces labels by the machine, and they **may not be human-readable**.
- **Ground truth** is a **properly labeled data set used as an objective standard** to train and assess a model. The accuracy of a trained model depends on the accuracy of your ground truth, so ground-truth quality is critical.

9.3 Data mining

Data mining is the **extraction of patterns and knowledge from large amounts of data**, *not* the extraction of the data itself. The industry process **CRISP-DM** describes six phases:

Phase	Question it answers
Business understanding	What does the business need?
Data understanding	What data do we have?
Data preparation	How do we organise the data for modelling?
Modelling	Which modelling techniques should we apply?
Evaluation	Which model best meets the business objectives?
Deployment	How do people access the results?

Data-mining methods

Ways of finding valid patterns and relationships in huge data sets, important for ML because a model is often trying to find a pattern or predict. The slide's methods are **classification, clustering, regression, sequential patterns, association rules, outlier detection and prediction**. Andrew highlights that **classification, clustering, regression and association** are the four that "show up again and again" in classical models (Chapter 3).

9.4 Knowledge mining

Knowledge mining is a discipline in AI that uses a combination of intelligent services to **quickly learn from vast amounts of information**, letting organisations deeply understand and explore information, uncover hidden insights and find relationships at scale. The term was coined at **Microsoft** (you hear it around Azure, less around AWS or Google Cloud), but the concept is portable. Importantly: **there is a lot of overlap with RAG**; in many cases RAG is knowledge mining.

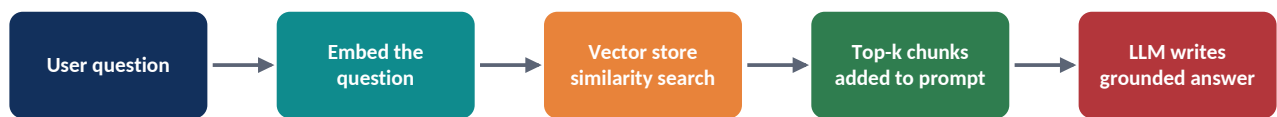


Figure 9.1: Retrieval-augmented generation follows the same ingest, enrich, explore idea as knowledge mining.

The three stages:

Stage	What happens	Examples
Ingest	Bring content from a range of sources using connectors to first- and third-party data stores	Structured: databases, CSVs. Unstructured: PDF, video, images, audio
Enrich	Add AI capabilities to extract information and patterns	Vision, language, speech, decision and search services. On AWS: Rekognition (vision), Polly (speech), Kendra (search)
Explore	Use the enriched data in applications	CRM, ERP, Power BI, and returned to an LLM to interpret, which completes RAG

9.5 Data wrangling

Data wrangling (also called **data munging**) is transforming and mapping data from one raw form into another format that is more appropriate and valuable for downstream purposes such as analytics. Andrew's shorthand: *pre-processing or cleaning your data for use in your ML models*. Six core steps:

1. **Discovery**: understand what the data is about, keeping domain details in mind.
2. **Structuring**: organise content into a structure that is easier to work with.
3. **Cleaning**: remove outliers, change null values, remove duplicates and special characters, standardise formatting.
4. **Enriching**: enhance the data with context from additional sources.
5. **Validating**: authenticate reliability, quality and safety.
6. **Publishing**: place the data in a data store for downstream use.

AWS tools named: **SageMaker Data Wrangler** and **AWS Glue DataBrew**.

9.6 Data modelling and data analytics

A **data model** is an abstract model that organises elements of data and standardises how they relate to each other and to real-world entities. Levels:

Level	Meaning	Example
Conceptual	How data is represented at the organisation level, abstractly	People, orders, projects and their relationships
Logical	How data is presented in software	Tables and columns; object-oriented classes
Physical	How data is physically stored	Partitions, CPUs, table spaces

Data modelling is the process of defining and analysing the data requirements needed to support business processes. (Note this is a different sense of "model" from an *ML model*, which Andrew flags deliberately.)

Data analytics is about examining, transforming and arranging data to extract and study useful information. Analysts commonly use **SQL, BI tools and spreadsheets**. Typical workflow: **ingestion, cleaning and transformation, dimensionality reduction, analysis, visualisation**.

9.7 Data roles

The **data scientist** is a person with multidisciplinary skills in **math, statistics, predictive modelling and machine learning**, combining **computer science, maths/statistics and domain knowledge**. Missing one of the ingredients means you are "not necessarily a data scientist". Why each matters:

- **Computer science:** many ML models are based on algorithms.
- **Math and statistics:** classical ML relies heavily on statistics.
- **Software development:** you will write lots of Python; performance-critical work may need lower-level languages.
- **Research skills:** sourcing, cleaning, preparing and validating data.
- **Domain knowledge:** to build something useful in an industry.

Definitions vary by company, so expect different descriptions online.

Role	Focus as summarised in the course
Data mining	Gain knowledge about a data set and use it for learning or processing
Data wrangling	Convert and map raw data into a form fit for analytics and ML
Data analysis	Use existing information to uncover actionable insights for better decisions
Data scientist	Maths, stats, predictive modelling and ML to make future predictions
Data engineer	Infrastructure and architecture of data generation and movement; deploying ML models at scale or in a distributed architecture

9.8 Training, validation and test data sets

Data set	Purpose
Training	The data the model learns on
Validation	Checks the model is working correctly; used to tune hyperparameters during development
Test	Provides an unbiased evaluation of the final model after it has been trained

Ground truth (labeled-as-correct data) can appear in *all three*; the model does not "know" which answers are truth. It is used to test against the model.

WATCH OUT

Never tune on the test set. If the test set influences your choices it stops being an unbiased measure and you will overestimate real-world performance.

9.9 Corpus and corpus linguistics

A **corpus** (plural **corpora**) is a **large collection of naturally occurring text structured for analysis**. It can be as small as **50,000 words** or tens of millions. Sources: books, newspapers, magazines, transcripts, web pages. Examples: a company building an **English dictionary** needs a corpus of words; a company building **academic textbooks** could use a corpus of lecture and seminar transcripts.

Corpus linguistics is the study of language using corpora, performing statistical analysis, hypothesis testing, checking occurrences and validating linguistic rules. It looks for lexical and grammatical patterns to answer questions such as *What is the most frequently used word? How often are idioms used? Which words are used in formal situations?*

9.10 Data types

A **data type** is a particular kind of data item that serves a specific purpose so that operations can be performed with it. The ML-relevant tree:

	Qualitative (quality)	Quantitative (quantity)
Sub-type	Categorical : values are labels	Numerical : values are numbers
Finer	Discrete : countable and finite. Binary (two options, e.g. 0/1, true/false), Nominal (labels where order does <i>not</i> matter), Ordinal (labels where order <i>does</i> matter)	Continuous or discrete numbers

BEYOND THE VIDEO

Encoding turns categorical values into numbers for models (for example one-hot encoding for nominal data, integer ranks for ordinal data). That is the "encoding" step behind feature engineering (Chapter 6).

9.11 Self-check

1. What is ground truth, and why does its quality cap model accuracy?
2. List the six data-wrangling steps.
3. Which data set gives the final unbiased evaluation?
4. How does knowledge mining relate to RAG?
5. Ordinal or nominal: shirt sizes S/M/L? Colours red/green/blue?

Choosing Models: Leaderboards, Benchmarks and AI Assistants

Source: video part 0, 3:48-4:25 (leaderboard discussion with Rola Dali, Artificial Analysis, Hugging Face Open LLM leaderboard, LiveBench; and the "AI-powered assistants" tour) plus slide "Quality & Context Window".

10.1 Why compare models?

With dozens of models available, the practical question is *which one should I choose?* Andrew admits up front that he does not know the best way to evaluate models, so he invites Rola Dali to dig deeper than his earlier videos. Their conclusions form a useful decision method.

The sites they pull up:

Site	What it offers
Artificial Analysis	Independent comparison of many models on quality, price, speed (tokens per second), latency (time to first token) and context window , including per-provider views
Hugging Face Open LLM Leaderboard	Rankings of open models on a fixed set of benchmark tests
LiveBench	A benchmark whose questions are refreshed regularly
Provider pages (e.g. Anthropic's model family page, showing Opus / Sonnet / Haiku tiers)	The provider's own numbers

10.2 Reading a leaderboard such as Artificial Analysis

Take the example Andrew shows: **Claude 3.5 Sonnet** (with an October release and an earlier June release). The page summarises:

- **Quality:** a quality index compared with other models.
- **Price:** dollars per million input and output tokens (the page lists blended and separate prices).
- **Speed:** output tokens per second.
- **Latency:** time to first token (TTFT).
- **Context window:** the maximum number of tokens the model can handle (the slide shows a 200k example for that model).

The model is then plotted against others on quality, speed and price charts. Andrew notes that trade-offs are visible: a model can be higher quality yet more expensive with higher latency.

KEY IDEA

A model choice is a **three-way trade-off between quality, speed/latency and cost**, constrained by **context window** and by task fit.

10.3 Benchmarks: what they measure

A benchmark is a standard test set with known answers. The video makes several points:

1. **Match the benchmark to your task.** If you build a coding assistant, ignore MMLU and look at a coding benchmark such as **HumanEval**. If you want creative language (Andrew's example: a Japanese visual-novel generator) you need a benchmark about **creativity of natural language**, which is harder to find; there is no single standard list of benchmarks.
2. **Common benchmark families** include general knowledge and reasoning (**MMLU**), math, coding (**HumanEval**), language accuracy (which relates to **hallucinations**) and reasoning questions. Most leaderboards choose about six.
3. **Math matters** because it is the "Achilles heel" of LLMs: they generate language one token after another and have historically been weak at math and real-world understanding, such as physics.
4. **Be cautious about provider-run benchmarks.** They can be biased, intentionally or not.
5. **Benchmark leakage.** Test questions are supposed to be withheld. If they are scraped into training data the model may have seen the answers, making the benchmark inaccurate. **LiveBench** addresses this by **renewing its questions almost every month**.
6. **Leaderboards keep moving.** A paper is a single point in time; third-party leaderboards re-run tests at regular intervals so you can see how models compare over time.

WATCH OUT

A high score on a leaderboard is not a guarantee for *your* use case. Always test candidate models on **your own examples** before committing.

EXAM TIP

"Which factor does a benchmark leakage problem affect?" It makes benchmark scores **less trustworthy** because training data may contain the test answers.

10.4 The Chinchilla paper

While on Artificial Analysis Andrew asks about "Chinchilla". Rola explains that the **Chinchilla paper** (rather than the model) is the interesting part. It evaluated **model size versus data-set size versus compute** together, looking at all the resources involved in training. The lesson (revisited in the optimisation lab): **bigger is not simply better**. A large model that does not see enough data is not efficient; parameters, training tokens and compute must be balanced. Rola calls it a paper everybody should read.

10.5 Choosing a model: a practical procedure

1. **Define the task:** coding, creating language, reasoning, extraction, chat.
2. **Pick benchmarks that measure that task**, not just the headline average.
3. **Shortlist** by quality, then filter by **price, latency and context window**.
4. **Decide the deployment style:** managed API, cloud-hosted model, or local open-weight model (Chapters 13 and 15).
5. **Test with your own prompts** and a small evaluation set.

6. **Re-check periodically**; new models appear constantly.

Rola's summary of when to use GenAI: often the best tool is "the most cost-effective precise tool", and that is not always an LLM if you already have domain expertise. What GenAI is really good at, in her experience, is **getting started**: producing a draft of a report or paper so you can jump in and edit, and polishing.

10.6 The AI-powered assistants tour

Andrew then surveys the **AI-powered assistants** you can use without code, to give exposure to ones you may never have tried. Every paid tier tends to cost about **\$20 a month**, so "you can't use them all; pick a favourite".

Assistant	Notes from the video
ChatGPT (OpenAI)	Andrew's default: "not the best but generally works really well and just keeps going". Has multiple models (4o, o1) where some have usage limits. Handled a PDF upload, though extracted text from the <i>Attention Is All You Need</i> paper had encoding issues until it used OCR on an image
Gemini (Google)	Known for parsing documents well; PDFs may need Drive integration or an advanced tier. Read a PDF attachment without complaint
Meta AI	Powered by Meta's open Llama models
Mistral (Le Chat)	Powered by Mistral's own models; seems more a showcase of capability than a consumer product
Claude (Anthropic)	Named later as a strong coding assistant
Microsoft Copilot	Discussed with the coding assistants in Chapter 16
Chinese assistants (e.g. inside WeChat or Alibaba)	Exist but hard to access outside China, so not demonstrated

Andrew's points on how they behave: long conversations keep going because the assistant is likely **summarising context underneath** (the trick from Chapter 7), and different model tiers in the same product have different limits.

BEYOND THE VIDEO

AI assistants are the *product layer*. The rest of the course goes below them to APIs, playgrounds and local models, where you control parameters like temperature, top-k, system prompts and cost.

10.7 Self-check

1. Name three attributes compared on Artificial Analysis besides quality.
2. Why should you not pick a coding model based on MMLU alone?
3. What problem does LiveBench's monthly refresh solve?
4. What did the Chinchilla paper compare?
5. Why might a provider's own benchmark be biased?

Development Environments for GenAI Work

Source: video part 0, 4:25-6:10 (labs on Google Colab, SageMaker Studio Lab, Lightning AI, GitPod, Deepnote, Miniconda, VS Code, JupyterLab). Slide OCR for these labs was screen recordings, so the transcript is the main source.

11.1 The big picture

Before calling models or building apps you need somewhere to run code. Andrew's course tours **notebook services**, **cloud IDEs** and a **local set-up**, and keeps returning to one theme: pick what is *free, quick to start and matched to your compute needs*. If you only call **service APIs** (OpenAI, Anthropic, Bedrock and so on), the machine barely matters. If you run models locally, RAM, cores and GPUs matter a lot.

Option	Type	Good for	Caveats from the video
Google Colab	Hosted notebook	Very popular; free tier; tutorials everywhere	Odd UI in Andrew's opinion, but "you're going to run into it a lot". Colab Pro was about \$13.99 for 100 compute units a month, Pro+ for 500
Amazon SageMaker Studio Lab	Hosted notebook (free CPU and GPU)	Free compute; JupyterLab-style	You must sign up and request an account; do not confuse with the paid <i>SageMaker Studio</i> ; includes a Gradio example kernel
Lightning AI Studios	Cloud IDE	VS Code experience, notebooks and terminal in one; switch between CPU and GPUs (T4, A10G, L4, A100, H100)	Free sign-up with paid tiers; templates such as a language-model studio; GPU access is limited to deter crypto mining
GitPod	Cloud dev environment	Launches extremely fast from a GitHub repo; open VS Code in a browser	The vendor was changing its model at the time, so check current status
Deepnote	Hosted notebook	Free tier; can attach GPUs; built-in publishing to an app	Andrew tried it for the first time in the video
Local machine (Miniconda + VS Code / JupyterLab)	Local	Full control; free	Needs enough RAM (Andrew suggests 32 to 64 GB for local LLM work) and cores

11.2 Google Colab basics

A Colab notebook is created by logging in with a Google account and creating a **new notebook**. In the demo, Andrew checks the Python version with a shell command:

```
!python --version
```

The exclamation mark runs a shell command; `%` prefixes are "magic" commands for the notebook itself. Sessions can be terminated from the **Manage sessions** screen, which you should do to free resources.

11.3 SageMaker Studio Lab and Gradio

SageMaker Studio Lab provides free CPU/GPU compute. In Andrew's session he chose CPU and ran a **Gradio** example. Lessons:

- Environments matter: a specific **Gradio kernel** existed, and using the wrong kernel produced "no gradio" errors.
- If a kernel misbehaves, **restart the kernel** (the dot next to the kernel name shows that it restarted).
- Browser settings such as **JavaScript blocking** and CAPTCHAs can break sign-in.
- Running `launch(share=True)` or using the provided link gives a shareable public URL for a Gradio demo. A minimal app just takes a name and returns "Hello, name".

11.4 Lightning AI, GitPod and Deepnote

- **Lightning AI** lets you open a *Studio* from a model template (for example Llama 3.2 serving), then choose between **CPUs and GPUs** and how many. Andrew likes how it makes hardware selection explicit.
- **GitPod** hides compute levels but is very fast to launch: create a new file such as `example.ipynb` and start coding.
- **Deepnote** connects with a Google account (GitHub if you want to do more) and lets you run a BERT fine-tuning example; you can turn a notebook into an app.

WATCH OUT

Free notebook services change often: features move behind paid tiers, sign-up rules change, GPU availability varies. Treat the specific screens as examples of a *category*.

11.5 Local Python with Miniconda

Anaconda is a package manager plus trusted libraries commonly used in data science; **Miniconda** is its minimal installer. The value: manage **isolated environments** so different projects do not conflict on library versions.

Command cheat sheet (as used in the lab)

```
# create an environment (Python 3.10 chosen deliberately: "latest is not always best")
conda create -n hello python=3.10 -y

# activate it (you will do this again and again)
conda activate hello

# install a library from conda-forge (trusted community packages)
conda install -c conda-forge pytorch

# install from a requirements file (note the -r)
pip install -r requirements.txt

# list what is installed
pip list

# remove an environment completely
conda remove -n hello --all -y
```

Points made in the video:

1. **Python version matters.** Newer versions can break data-science libraries, so pick the version that works for your stack (3.10 in the lab).
2. **Check which Python you run** with `which python` / `where python`; on some systems you must use `python3`.
3. **conda-forge** is where you search for packages; you can also install with pip if a package is missing there.
4. The `python-dotenv` package loads **environment variables** from a `.env` file (`load_dotenv()`), used constantly for API keys. Never commit `.env` files.
5. Notebooks in VS Code need the `ipykernel` package in the environment. Andrew recreated the environment from scratch (`conda create -n hello python=3.10 ipykernel -y`) to prove the fix.

11.6 JupyterLab server

As a different experience from VS Code, install **JupyterLab**, start the server, and paste the **token** shown in the terminal (or set a password). Extensions are best installed with the documented method (`pip install ...`), then enabled in settings. Andrew found themes there and noted that some extension installs need a restart of the server and a browser refresh.

11.7 Choosing quickly

If you...	Use
Just want to try code now, free	Colab or Deepnote
Need a GPU for a demo	Lightning AI or SageMaker Studio Lab (free options)
Want a real dev experience	VS Code + Miniconda locally, or GitPod
Only call APIs	Anything; even a laptop

EXAM TIP

You may see a question on the difference between **Jupyter Notebook**, **JupyterLab** and **JupyterHub** (Chapter 2), or on why virtual environments are used (dependency isolation). Know both.

Prompt Engineering

Source: video part 1, ~6:05-7:00 (slides "Zero-shot / Few-shot", "Prompt Chaining", "Chain of Thought", "Tree of Thought", "CO-STAR", and the ReAct build-along; slide OCR confirms the titles Prompt Chaining, Tree of Thought and CO-STAR).

12.1 What is prompt engineering?

Prompt engineering is the practice of writing and structuring the input to an LLM so it produces the result you want. Andrew makes three honest observations:

1. **You already use these techniques without realising it.** While building a Japanese-subtitle tool he was using prompt chaining and state-and-transition management before he knew their names. His advice: *do not stress about memorising every technique*; know that they exist and come back to review them.
2. **Prompt engineering vs prompt designing** is a debated distinction. Some call plain prompt writing "prompt design" and reserve "engineering" for orchestrating larger systems with code. Andrew treats it all as prompt engineering.
3. There are **so many techniques** and websites; the *Prompt Engineering Guide* is a resource he likes.

12.2 Zero-shot prompting

In **zero-shot** prompting you give the model a task with **no examples**. It can do the task because of knowledge built in during training. In the demo, a sentiment-analysis prompt is answered with a label such as *neutral*, even though no examples of positive, negative or neutral were supplied.

```
Classify the sentiment of the text as positive, negative or neutral.  
Text: "The delivery arrived on Tuesday."  
Sentiment:
```

12.3 Few-shot prompting (in-context learning)

In **few-shot** prompting you include **examples** inside the prompt so the model can predict the next one from the pattern. Andrew's example: a model that is not good at sentiment analysis is given a series of tweets labelled negative or positive, then a new tweet. Because the examples are supplied **in the context of the prompt message**, this is also called **in-context learning**: the model is not retrained, it only reads examples at that moment.

```
Tweet: "I love this phone!"      Sentiment: positive  
Tweet: "Battery died in an hour" Sentiment: negative  
Tweet: "Best purchase this year" Sentiment:
```

KEY IDEA

Zero-shot = no examples. **Few-shot** = a handful of examples in the prompt (in-context learning). Neither changes model weights.

12.4 Prompt chaining

Prompt chaining means the **output of one LLM prompt becomes the input of the next prompt** in a sequence. The reason: when a task is too large or complex for a single prompt (or the context window is too small), you **break it into smaller tasks**.

Andrew's real example, from the GenAI Bootcamp: turn a Japanese movie's **subtitle file** into English plus grammar notes in a JSON lines file. One giant prompt produced mistakes and bad information. The chain that worked, run **per line** of the subtitle file:

1. Translate the line from Japanese to English.
2. List the grammar rules found in the sentence.
3. Explain each grammar rule and give five examples.
4. Format the original line, translation and grammar rules into JSON.

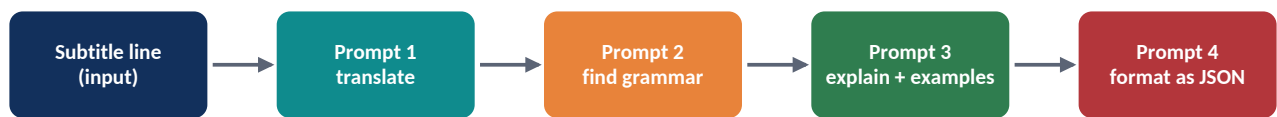


Figure 12.1: Andrew's subtitle chain. Each prompt's output becomes the next prompt's input.

12.5 Chain of thought (CoT)

Chain-of-thought prompting tells the model to **reason step by step** before answering. Without it: "Which two prime numbers sum to 10?" With it: "Please solve the problem step by step. First outline your reasoning as a clear, logical chain of thought, then provide the final concise answer."

Larger modern models need this less, but **smaller models** often do far better if they "think out loud". Andrew's point is simple: sometimes just telling the model to think step by step gives a much better result.

12.6 Tree of thought (ToT)

Tree-of-thought prompting instructs the model to **explore possible states and transitions** instead of committing to a single line of reasoning: effectively creating a **decision tree** with branches that can be added or removed. A widely reproduced diagram compares four styles:

Style	Idea
Input-output (IO) prompting	Input in, output out (zero-/few-shot)
Chain of thought	One linear line of reasoning
CoT with self-consistency	Generate multiple reasoning chains and use voting to pick the final answer
Tree of thought	Branch, evaluate options, prune and expand the best

Trying it out

Andrew looked for a template on **PromptHub** (most online examples are the *Game of 24* and creative writing). The version that worked used a **single prompt** that asks the model to: (1) restate the problem, (2) generate possible paths, (3) evaluate the branches, (4) expand the best branch, (5) give the final answer. On "What is 15 x 14?" it listed approaches (quick mental split vs. long multiplication), judged them, and expanded the best.

Insights from the session:

- A capable model may already "know" ToT: a short instruction "use tree of thought" worked. A **smaller model** may need the steps **explained inside the prompt**.
- ToT can be **one prompt** (a reasoning framework) rather than several chained prompts.
- In a movie-night example (three friends' tastes), the model wrote steps: understand each person's preferences, consider all three, score options, select the best.

12.7 CO-STAR framework

CO-STAR is a prompt template (created by Sheila Teo, according to the video). The mnemonic is Context, Objective, Style, Tone, Audience, Response.

Letter	Meaning	Example from the slide
C	Context: what are you doing?	A free online introductory Japanese workshop for beginners
O	Objective	Encourage people to register by highlighting the quick, fun, interactive nature of the session
S	Style	Engaging, inviting, clear, educational
T	Tone	Persuasive yet friendly
A	Audience	Busy adults curious about language learning with no prior experience
R	Response format	A concise, impactful Facebook post

It is "nothing super fancy", just an easy-to-remember way to structure prompt documents. Andrew uses something similar.

12.8 ReAct prompting (reason + act)

ReAct combines **reasoning** with **acting** (using tools). The model writes its thought, chooses an action, receives an observation, and repeats until it can give a final answer.

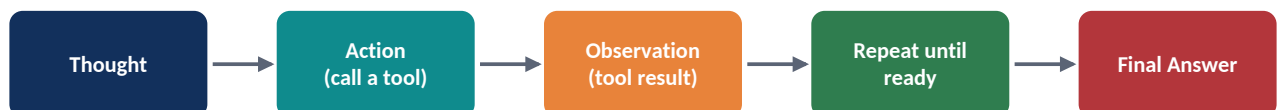


Figure 12.2: The ReAct loop.

Andrew built it **from scratch** rather than using LangChain or LlamaIndex, because he feels frameworks add a layer that does not help much when learning. The build (in GitHub Codespaces with ChatGPT's help):

1. **System prompt** instructing the model: you are an AI assistant; reason in a **Thought** section; when you need a tool output an **Action** with the tool name and inputs; after the tool call you will see an **Observation**; continue the cycle; end with a concise **Final Answer**. The chain of thought is for you, not the user.
2. A **single manual step** with the user prompt "What is the capital of France plus the capital of Germany?" gave: *Thought: ... Action: no tools needed ... Observation: Paris and Berlin ... Final Answer: Paris and Berlin.*
3. To test tools he asked for the **weather in his hometown**. The model produced an action but had not really run anything, so he added **Python code** to: check whether the model produced an action, **parse out the tool name with a regular expression**, run the tool in Python (an **OpenWeatherMap** API call), **append an observation message** to the conversation and call the LLM again.
4. Practical snags: API keys and organisation/project IDs belong in a **.env** file (added to **.gitignore**); an OpenWeather key can return **401 Unauthorized** until the account is confirmed; the weather endpoint wanted **latitude and longitude** instead of a city name; indentation errors in generated code.
5. He notes the system prompt should tell the model **exactly which tools exist**; otherwise it invents them.

```

loop:
    reply = llm(system_prompt + conversation)
    if reply has Action:
        tool, args = parse(reply)      # regex on "Action: ..."
        result = run_tool(tool, args)  # your Python code
        conversation += "Observation: " + result
    else:
        return reply.final_answer

```

KEY IDEA

ReAct = Thought, Action, Observation, repeat, Final Answer. Frameworks (LangChain, agent libraries) automate it, but knowing the manual moving parts makes them easier to use.

12.9 Choosing a technique

Situation	Technique
The model already knows the task	Zero-shot
Format or label style must be shown	Few-shot
Task is too big for one prompt	Prompt chaining
Multi-step logic or maths	Chain of thought (add self-consistency for more reliability)
Several possible solution paths	Tree of thought
You need repeatable prompt structure	CO-STAR
The model must call tools/APIs	ReAct

EXAM TIP

Match a description to a technique. "Output of one prompt is the input to the next" is **prompt chaining**. "Examples inside the prompt" is **few-shot / in-context learning**. "Multiple reasoning chains and voting" is **self-consistency**. "Explores branches like a decision tree" is **tree of thought**. "Thought, action, observation" is **ReAct**.

BEYOND THE VIDEO

Good prompts also state the **role**, give **constraints** (length, format such as JSON), include **delimiters** around user-supplied text, and ask the model to say "I don't know" when unsure, which reduces hallucination. Always test prompts against a small set of examples.

12.10 Self-check

1. Why is few-shot prompting called in-context learning?
2. When would you break a task into a prompt chain?
3. What does self-consistency add to chain of thought?
4. Expand the CO-STAR acronym.
5. In ReAct, what is added to the conversation after a tool runs?

Calling Model APIs and Cloud AI Platforms

Source: video part 1, ~6:55-8:40 (labs: OpenAI, Anthropic Workbench, Cohere, Amazon Bedrock, Google Gemini API and Vertex AI, Azure AI Foundry). Slide OCR shows dashboard screenshots; the transcript supplies the detail.

13.1 Two ways to use a model

The labs work through the same task, "send a prompt and read the reply", on many platforms. It helps to see them in two groups:

Group	Examples	You manage	Typical cost model
Direct provider APIs	OpenAI, Anthropic, Cohere, Google Gemini API	An API key , the request code	Pay per token (some free tiers or trial credits)
Cloud AI platforms ("models as a service")	Amazon Bedrock, Google Vertex AI, Azure AI Foundry	Cloud account, region, model access or deployment, IAM	Pay per token, sometimes provisioned capacity

Because you call a hosted model, **no GPU or big machine is needed**; a small notebook is enough.

13.2 The universal pattern

Almost every lab follows the same recipe. Learn it once:

1. **Create an account** and an **API key** (or cloud credentials).
2. **Store the key in a .env file**, add **.env** to **.gitignore**, and load it with **python-dotenv** (Chapter 11). Never paste keys into notebooks that will be committed.
3. **Install the SDK** (`pip install openai, anthropic, cohere, google-generativeai, boto3, ...`). Use **-q** for quiet installs and put **%** or **!** in front inside a notebook.
4. **Create a client**, choose a **model name**, send **messages**, read the **response**.
5. **Track cost**: tokens in and out, and delete or stop anything that keeps billing (deployments, runtimes, instances).

```
import os
from dotenv import load_dotenv
load_dotenv()

from openai import OpenAI
client = OpenAI() # reads OPENAI_API_KEY from the environment
resp = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "system", "content": "You are a concise assistant."},
              {"role": "user", "content": "Explain tokens in one sentence."}],
)
print(resp.choices[0].message.content)
```

KEY IDEA

A chat request is a **list of messages with roles**: **system** (rules and persona), **user** (the question), **assistant** (earlier replies). Sending the **whole conversation each time** is how the model "remembers" (it has no memory between calls).

13.3 OpenAI

Andrew sets up the OpenAI API with an **API key**, plus an **organisation ID** and **project ID** (found under API settings, *Manage projects* and *General*). Billing points from the lab:

- Prepaid **credits** are bought in advance and **they lapse after a year**, so use them.
- OpenAI also offers a **Playground** with a chat mode and many settings; the course mostly works *programmatically* but says the playground is a valid way to explore.
- Two IDs (project, organisation) are optional in many cases but are sometimes needed to route requests.

13.4 Anthropic and the Workbench

The **Anthropic Workbench** is Anthropic's playground for writing and testing prompts. Steps and lessons:

- Create an API key, store it in `.env`, `pip install anthropic`, and call the messages API with a model name.
- A first request without any content fails with "at least one message is required": the API insists on a **messages list**.
- There is no obvious free tier; Andrew added \$5 of credit (auto-reload off) to try it. API use is under **commercial terms**, and sign-up asks about intended use (for example, no products aimed at under-18s).
- The docs' **Ask AI** feature listed beta features: **computer use** (the model controls a desktop) and **message batches** (cheaper, offline processing), which may already have left beta. *Always check the current docs; APIs change.*

13.5 Cohere

Cohere offers a "generous free tier". Install with `pip install -q cohere`, copy the code sample from the dashboard chat page (top-right **Code** button), paste your API key and run. Points made:

- Its generative models (for example **Command R**) and its **embed** and **rerank** models are separate products (Chapter 7 and Chapter 20).
- Some Cohere models can also be **downloaded from Hugging Face** in another format.
- You still need a little extra code to format the response nicely.

13.6 Amazon Bedrock

Amazon Bedrock is AWS's models-as-a-service, "a collection of things you can do with LLMs". First-time use:

1. Pick a **region** (Andrew used N. Virginia).
2. Open the **Model catalog** and, at the bottom of the left menu, **Model access; request or enable access** to each provider's models.

- Launch a notebook. He used a **SageMaker Studio Lab-style JupyterLab** with small storage and **no powerful compute** because nothing is downloaded.
- Call models with the **Converse API** (via **boto3**), for example Amazon's **Nova Micro** (a low-cost model he had not used before).

Lessons: the API rejected his call with *invalid type for parameter system*: **the `system` field must be a list of blocks, not a plain string**. Read the error, fix the shape, run again.

13.7 Google Gemini API and Vertex AI

- Gemini API (AI Studio)**: `pip install google-generativeai`, load the key from `.env`, call the model. The install line needs the `!` or `%` prefix in a notebook.
- Vertex AI Studio (Google Cloud)**: choose a model from **Model Garden**, and click **Enable**. The console shows pricing immediately (about **\$1 per million tokens** for the example) and you must agree to the cost. You can **open the sample directly in Colab Enterprise**, then delete the notebook and the **runtime** afterwards to stop charges.
- Google collects its separate image, speech and other generation services under Vertex AI.

13.8 Azure AI Foundry (Azure OpenAI)

Azure's flow is different again:

- Create a **project** in **Azure AI Foundry**, then **deploy a model**, choosing a deployment type. **Global standard** is pay-per-token; **batch** is offline (you cannot infer with it immediately). Andrew guesses global might be pricier than standard but says this is not always so.
- After deployment, use the **playground** or the sample code; the SDK now uses a **project client** and changes frequently ("APIs are always changing, folks; I was doing this three months ago and it's completely different").
- Deleting a deployment** is done by clicking into the model in the portal and deleting it there. Delete deployments and compute instances when finished; keep the project if you will reuse it for Azure ML.

Platform	You "enable" via	Code entry point
Bedrock	Model access page	boto3 Bedrock runtime (Converse)
Vertex AI	Enable model in Model Garden	Vertex SDK / Gemini API
Azure AI Foundry	Create a deployment	Azure AI project client / OpenAI-compatible endpoint

13.9 Playgrounds and workbenches

A **playground** (OpenAI Playground, Anthropic Workbench, Bedrock playgrounds, Vertex AI Studio, Azure playground) is the browser UI to test a model **before writing code**. Roadmap talk (Chapter 1) noted these expose more knobs than chat assistants: **temperature**, **top-p / top-k**, **max tokens**, **stop sequences** and the **system prompt**. Learn one and the others feel the same.

Setting	Effect
Temperature	Higher = more random/creative; lower = more deterministic
Top-p / top-k	Limit the pool of candidate next tokens

Setting	Effect
Max tokens	Cap on output length (counts against the context window)
System prompt	Persistent instructions that set behaviour

13.10 Cost and hygiene checklist

- Prefer **small, cheap models** for development and testing.
- **Delete deployments, endpoints, runtimes and notebooks** you are not using.
- Set **budgets and alerts** in the cloud console.
- Keep **API keys out of Git**; rotate any key you exposed.
- Expect **API and SDK changes**; keep the docs open.

EXAM TIP

Know the difference between calling a **hosted model through an API** (no infrastructure, pay per token) and **self-hosting** (you manage hardware, Chapters 15 and 18). Also know that **Bedrock, Vertex AI and Azure AI Foundry** are the AWS, Google and Microsoft equivalents.

13.11 Self-check

1. Why is no GPU required for these API labs?
2. What are the three message roles in a chat request?
3. Where should API keys be stored during development?
4. What did the Bedrock "invalid type for parameter system" error teach?
5. Which Azure deployment type is pay-per-token and which is offline?

Hugging Face: Models, Pipelines, Spaces and Datasets

Source: video part 1, ~8:40-10:10 (labs on Hugging Face, Llama 3.2 on Lightning AI, **pipeline**, Spaces including a 3D-asset generator, datasets and a small fine-tune). Roadmap slide: Hugging Face is "the GitHub of AI/ML".

14.1 What is Hugging Face?

Hugging Face (HF) is the central hub of the open model ecosystem. In the roadmap discussion (Chapter 1) it was called "the GitHub of AI/ML". It hosts:

Part	What it is
Models	Hundreds of thousands of model repositories (weights, tokenizer, model card)
Datasets	Ready-to-load training and evaluation data sets
Spaces	Hosted demo apps (Gradio, Streamlit, Docker) you can try or copy
Libraries	transformers , datasets , accelerate , sentence-transformers , diffusers , and more
Inference	Hosted endpoints and providers for running models

A **model card** describes the model, licence, intended use and often includes copy-paste code. Andrew's note: some model pages require you to **accept a licence** or **request access** (for example Meta's Llama models) and you need an **access token** to download them.

14.2 Running a model on a GPU studio

To run **Llama 3.2** he uses **Lightning AI** (Chapter 11) because it "is so easy to start" and has templates. Key workflow:

1. Start a Studio, pick **CPU or GPU** hardware.
2. Confirm the **conda environment** is active (**conda info** shows the path). Something that works in one terminal may not work in the notebook if it is a different environment: check where you are running.
3. **pip install transformers torch accelerate** and load the model.
4. Store the **Hugging Face token** in **.env**, and load it with **python-dotenv**.

Errors and fixes seen along the way:

Error	Meaning / fix
"Please run pip install accelerate "	A required library is missing; install then restart the kernel
"No matches found for transformers, torch"	Shell quoting issue; run the install on its own line and restart
Model path / folder does not exist	A local directory was expected; specify the model id or download first

Error	Meaning / fix
Missing tokenizer dependency (e.g. sacremoses for some tokenizers)	Install it and restart the kernel

14.3 The **pipeline** abstraction

The **pipeline** function in **transformers** is a **wrapper around task-specific pipelines** that hides tokenization, the model call and post-processing. It is described as "a great way to ensure models for inference": less fiddling, fewer places for error.

```
from transformers import pipeline

# Text classification (sentiment)
clf = pipeline("sentiment-analysis")
print(clf("Great service, the food was delicious"))

# Text generation
gen = pipeline("text-generation", model="meta-llama/Llama-3.2-1B-Instruct")
print(gen("Explain embeddings in one line.", max_new_tokens=60))

# Summarisation
summ = pipeline("summarization")
print(summ(long_text, max_length=80, min_length=30))
```

Tasks shown or named: **text classification**, **text generation**, **summarisation** (Andrew asked an assistant for a long passage to test summarisation, "not a good summarisation task" for the short sample).

Pipeline vs direct model control

	pipeline	Direct model control (AutoTokenizer , AutoModelForCausalLM)
Effort	Minimal code	More code, more settings
Flexibility	Limited	Full control over generation, chat templates, batching
Errors	Fewer	More: config files and input parameters may need tweaking

Andrew's example: running a chat with Llama 3.2 directly, on a **limited-CPU** machine, worked but RAM and CPU usage were visible; when an error appeared he had to look for a configuration change or a parameter fix. "If you use the pipeline there's less fiddling around, but if you use direct model control you have more control."

KEY IDEA

Small models run on CPUs; a **1B parameter** model needs a few GB of memory, while **70B** models need a lot more (Chapter 19). Use the **smallest model that solves the task**, as the roadmap talk stressed with BERT.

14.4 Hugging Face Spaces

Spaces host demo apps. Andrew tries a **3D-asset generator**: upload an image, click *Generate*, and a **3D model** (GLB file) is produced. Observations:

- If the image has an **alpha channel** it is used as a mask.
- The Space ran on **ZeroGPU**, a shared-GPU hardware type for Spaces.
- Generation took about **42 seconds**; the file could then be extracted as **GLB**, opened in a viewer, or brought into **Blender** and **low-poly'd** for game use.
- Similar services exist elsewhere: **Replicate** lets you find a public model, run it in a web interface, or call it by API (covered later).

This connects to the **modalities** discussion: 3D is a GenAI modality alongside text, image, video and audio.

14.5 Datasets and a small training run

The **Datasets Hub** lets you load data with **datasets**. Andrew opens **SmolTalk**, a synthetic data set for **supervised fine-tuning** (used for the SmolLM2-Instruct models), and then builds a notebook that:

```
from datasets import load_dataset
ds = load_dataset("some/dataset")           # e.g. an emotion/sentiment set
```

1. Loads a data set from HF.
2. Brings in **BERT base uncased** (Chapter 5) and its tokenizer.
3. Creates a **tokenized data set**, prepares a **Trainer** with **training arguments**, and runs a **training loop**.
4. Notes he wants to save the resulting model and reuse it.

He reminds viewers of Rola's advice: "**you don't always need an LLM**"; a small model like BERT is often the cost-effective tool, which is what the notebook demonstrates. Andrew also switched between **VS Code** and **JupyterLab** in the same session ("I'm indecisive; I switch between both all the time").

14.6 Putting it together: fine-tuning workflow on HF

BEYOND THE VIDEO

The video shows the pieces; this is the full order of operations you would follow.

1. **Pick a base model** (small if possible) and check its licence.
2. **Load and inspect a data set**; clean and split it (train / validation / test, Chapter 9).
3. **Tokenize** with the model's own tokenizer.
4. **Configure training** (learning rate, epochs, batch size are hyperparameters, Chapter 6).
5. **Train** (use PEFT/LoRA for big models, Chapter 8).
6. **Evaluate** on the test set.
7. **Push to the Hub** or save the weights, and write a model card.

14.7 Exam-style takeaways

EXAM TIP

Hugging Face = hub for models, datasets and Spaces; **transformers.pipeline** = highest-level API for inference; you need the **same tokenizer as the model**; some models are **gated** and need a token; Spaces are demo apps; **ZeroGPU** is shared GPU hardware for Spaces.

14.8 Self-check

1. What is a model card and why read it?
2. Give two advantages of `pipeline` over direct model control.
3. Why does the notebook sometimes need a kernel restart after `pip install`?
4. What are Spaces used for?
5. Which library loads datasets from the Hub?

CHAPTER 15

Running Models Locally, Frameworks and Structured Output

Source: video part 1, ~10:10-11:50 (labs and slides on Ollama, Open WebUI, llamafile, LangChain, LlamaIndex, llama.cpp, BitNet, context/prompt caching, structured JSON, Instructor, Docker).

15.1 Why run a model locally?

The roadmap question "cloud or local?" (Chapter 1) is answered by trade-offs:

	Hosted API (Chapter 13)	Local / self-hosted
Cost	Pay per token	Hardware and electricity; free per token
Privacy	Data leaves your network	Data stays with you
Setup	Minutes	Downloads, drivers, RAM/GPU needs
Quality	Best frontier models	Open-weight models (Llama, Mistral, Gemma, Qwen...)
Control	Provider limits and changes	Full control, pinned versions

Andrew ran many of these labs on an **Intel Lunar Lake developer kit** (a machine with more compute for local models) reached remotely, plus WSL Ubuntu, Lightning AI and GitPod. Local model work needs **RAM** (roughly 32-64 GB is comfortable) and, ideally, a GPU or accelerator.

15.2 Ollama

Ollama is the easiest way to **pull and serve models** on consumer machines (laptops and desktops). It runs a server (default port **11434**) and exposes commands:

```
ollama pull llama3.2:1b    # download a model
ollama run llama3.2:1b    # chat in the terminal
ollama list                # what is downloaded
ollama rm llama3.2:1b     # remove a model
ollama serve              # start the server (it may already be running)
```

Lessons:

- If **ollama serve** says the port is in use, **Ollama is already running** in the background (a very common confusion).
- You can call it programmatically at **http://localhost:11434** using its own API or an **OpenAI-compatible endpoint**, so client code for OpenAI works with a changed base URL.
- Andrew says Ollama is "so darn easy" that you should just spend a little time learning to pull and serve models. He finds it "too basic" for his own daily work but recommends it for learning.
- It can be combined with **LangChain**, containers, and UIs such as **Open WebUI**.

Open WebUI

Open WebUI is a browser chat interface that connects to Ollama. It is usually run as a **Docker container** (port 3000) and needs access to the Ollama host. In the lab Andrew fought several Docker problems (see 15.8). Its value: a ChatGPT-style front end for local models, with model selection and conversation history.

15.3 OPEA and Intel

Andrew mentions **OPEA** (Open Platform for Enterprise AI), "a framework for running GenAI workloads in containers", useful if you run GenAI on a company machine without much coding. It reappears in the deployment chapter with Intel hardware (Chapter 18).

15.4 llamafile

llamafile (from Mozilla) packages a model and runtime into a **single distributable executable file**. Steps:

1. Download a llamafile for a chosen model (Andrew preferred **Llama 3.2 3B** over the older 1.5B example).
2. **Windows**: rename the file with an **.exe** extension. **Mac/Linux**: make it executable (**chmod +x file**).
3. Run it; it opens a chat UI and serves an API on **port 8080**.

The lab notes the model inside is a **GGUF** file. He admits he "hasn't had many use cases" for it, but it is fun and works with no installs.

15.5 llama.cpp and GGUF

llama.cpp is an **inference engine** that reimplements the Llama architecture in **C/C++** for high performance on **CPUs** (and other hardware), with minimal setup. Key facts:

- It works **only with GGUF** model-weight files.
- It offers a **CLI tool**, a **lightweight server**, and **bindings for almost every language** (Ruby was the slide example).
- Installation options include Homebrew on Mac and Linux; Andrew eventually used **GitPod** because it has Homebrew and gave a high chance of success after repeated failures elsewhere ("I've tried this video like nine times").
- Practical details: the copied command had a **missing backslash**, so the flags failed until fixed; the tool downloads the model into a **cache** directory; **verbose mode** floods the output (turn it off); **"echo the prompt"** flags change what you see.
- Interactive mode lets you set a system message such as "You are a helpful assistant" and chat.

KEY IDEA

GGUF is the single-file format used by llama.cpp, Ollama and llamafile to store quantized weights plus metadata. Hugging Face hosts many GGUF versions of popular models.

BitNet

BitNet (Microsoft's **bitnet.cpp**) pushes quantization to the extreme: weights reduced to about **1 bit** (values like -1, 0, 1; the transcript refers to "the smallest value possible"). Compared with an already optimised llama.cpp, the slide claims large speed and energy gains (**about 55-70%**, depending on model). The limitation: only certain models exist for it (a BitNet b1 Llama variant and the **Falcon 3** family).

15.6 LangChain and LlamaIndex

Framework	What it is	Strength	Andrew's view
LangChain	Opinionated framework to build LLM apps and agents with one API over many models	Huge ecosystem, agents, chains, integrations	An extra abstraction that "does not always pay off in the long term"; not designed for scale on a server
LlamaIndex	Open-source framework to prototype agents and LLM workloads	Wider data connectors and advanced RAG techniques; data indexes, query engines, agents, observability, evaluation, workflows; LlamaCloud for enterprise	The two are near parity; same caveat

He mentions both because you might like them for learning, since they ship predefined agents and workflows you can study, then "manually convert that over and build that yourself". Being an "opinionated" framework can be good for starters but there is risk in buying into a vendor's platform (LangChain's LangSmith/LangGraph platform, LlamaCloud).

15.7 Context (prompt) caching

Context-window caching, also called **prompt caching** or **context caching**, stores the **computed context in memory** so a later request that reuses the same prefix answers faster and cheaper. Good uses:

- Chatbots with **extensive system instructions**
- **Repetitive analysis of lengthy video files**
- Recurring **queries against large document sets**
- **Frequent code-repository analysis** or bug fixing

Points from the slide:

- Offered by **some providers** for **specific model versions** (Andrew mentions Google Gemini and Anthropic as two providers).
- Implementing it yourself is very hard: you would have to understand the model's architecture, separate the tokenization step and inject data afterwards ("you'd have to mangle a model").
- Benefits: **faster responses on large inputs** and **lower cost** (cached tokens may be billed at a reduced rate; check each provider).

15.8 Docker on Ubuntu: lessons from the lab

Docker consumed a large part of the session, and the fixes are broadly useful:

Problem	Fix
Installed Docker with snap ; awkward and needs sudo every time	Remove it (sudo snap remove docker) and install from the official Docker apt repository instructions
"Permission denied" on the Docker socket	Add your user to the docker group (sudo groupadd docker, sudo usermod -aG docker \$USER , then log out/in or newgrp docker so it takes effect)

Problem	Fix
Cannot stop a container	<code>docker ps</code> to find it, then <code>docker stop <id></code> or <code>docker kill <id></code>
Open WebUI cannot see Ollama	Use host networking (run with <code>--network=host</code>), instead of mapping to an internal IP

The docker `-d` flag runs a container **detached** in the background; that is how you know it is "still running".

15.9 Structured output (forcing JSON)

Many apps need the model to return **valid JSON**, not prose. Andrew's experience: **just asking the model for JSON is unreliable**; you need a step **outside the LLM** to force the shape.

Techniques

Technique	Idea
Context-free grammar	Constrain token choices to those that fit a grammar
Finite-state machine	Track valid next tokens as states
Regular expression	Reject tokens that break the pattern

The mechanism (as explained): the tool checks each candidate **token by token**. If the first token is not `{` or `[`, it is thrown away until a token matches what the pattern allows, so the model is *forced* to produce exactly the format wanted.

Where it comes from

- **Built into the model API:** OpenAI's structured outputs use **Pydantic** (a data-validation library for Python) to define the schema. Cohere tells you to explicitly ask for JSON in the prompt.
- **Third-party libraries:** **Instructor** was the one that "worked perfectly" for Andrew (docs at python.useinstructor.com), demonstrated with **Groq**; he wished it supported Amazon Bedrock at the time.
- Caveats: vendor features are often **beta** and may vanish; reliability sometimes only comes from **specific models** (OpenAI's).

```
from pydantic import BaseModel
import instructor
from openai import OpenAI

class Person(BaseModel):
    name: str
    age: int

client = instructor.from_openai(OpenAI())
person = client.chat.completions.create(
    model="gpt-4o-mini", response_model=Person,
    messages=[{"role": "user", "content": "John is 30 years old."}]
)
print(person)          # Person(name='John', age=30)
```

EXAM TIP

Structured output = making an LLM return a **defined data structure (usually JSON)** using grammars, state machines or regex, or a library like **Instructor** with **Pydantic** schemas.

15.10 Self-check

1. Why does "port already in use" often appear when you run `ollama serve`?
2. What file format do `llama.cpp`, Ollama and `llamafile` use?
3. Name two benefits of prompt caching and two ideal use cases.
4. Why is "please output JSON" not enough for reliable structured data?
5. What problem does adding a user to the `docker` group solve?

AI Coding Assistants

Source: video part 2, ~11:50-14:10 (labs comparing GitHub Copilot, Windsurf (Cascade), Cursor, Amazon Q Developer and others on the same task). OCR of the editor screens is noisy; the transcript carries the detail.

16.1 The task: a "Linktree clone"

To compare assistants fairly, Andrew gives each one the same job: **build a Linktree clone** (a page of links for a profile) with a **Go API backend** and a simple front end, then add features (a **SQLite** database, seed data, and a README with commands). Because the goal is *workflow exposure*, he repeats it in several tools and notes their strengths and annoyances.

The category is called **AI-powered coding assistants** (or AI code editors). They sit between chat assistants (Chapter 10) and fully autonomous agents (Chapter 21):

Type	Description	Examples in the course
Autocomplete / extension	Suggests code inline inside your editor	GitHub Copilot, Amazon Q Developer, Gemini Code Assist
AI-native editor	A fork of VS Code with chat, edits and "agent" modes built in	Cursor, Windsurf (formerly Codeium)
Cloud IDE assistants	AI built into an online environment	Codespaces + Copilot, Lightning AI

16.2 GitHub Copilot

- Works inside **VS Code** (and other IDEs). Chat opens beside your code; **inline chat** (right-click, "Copilot") edits the selected code, for example "this code should pull from an SQLite database".
- The model can now be **switched** (Andrew was curious how the choice changes results).
- His view: the code Copilot produces is "not as good as ChatGPT or Claude" but it is convenient and integrated. It produced a **large amount of code** (routes, initialisation) even for small asks.
- Workflow points: open the project folder, **clone the repo**, rename folders carefully (module names matter in Go), navigate to the right directory before running **go** commands, and create a **README** with steps: **go mod init**, **go mod tidy**, then run.

16.3 Windsurf and Cascade

Windsurf is an AI-native editor from Codeium. Its agent, **Cascade**, is the standout in Andrew's session:

- After allowing access and signing in, you chat with Cascade in the project folder. It understood a request for a **Link tree clone with a Go backend and a simple front end**, inspected **main.go**, and **generated a front end**, which he found "super cool".
- It suggests **terminal commands** (you accept or reject), shows which files it edited, and can **run** steps for you.
- **Credits**: the free plan offers a limited number of **premium "user prompt credits"**; using a premium model (Cascade) costs one credit per use, and the pricing page compares a few hundred vs thousands of credits per

month. Andrew says it "does such a good job working with Cascade in such few lines".

- Errors are normal: a **SQLite dependency version** (module requires Go 1.16) broke the build; he asked the tool to fix it and, after a couple of tries, noted "now we're just wasting our credits". Reading the error and pinning the version fixed it.
- He felt he had not used its full potential, and that in languages like Python or Ruby the editing commands would shine.

16.4 Cursor

Cursor is another AI-native fork of VS Code.

- **Shortcuts:** **Ctrl+K** edits code in place; **Ctrl+L** opens **chat** (Andrew had to look these up in the docs: "if you don't have the tutorial from the beginning, you wouldn't remember").
- You can **change the model** in the chat box, and use **agentic mode** so it creates files and runs commands.
- Request used: "use SQLite as the database, and create a README with a bash command to seed the database." Cursor changed dependencies and asked him to **accept** changes; the first run still failed on the same dependency issue.
- His feedback: he likes the **right-click context menu** of other editors (Amazon Q, Copilot) and dislikes hunting for options. Also: assistants sometimes write files into the wrong directory because context is **per repository**, not per folder.

16.5 Amazon Q Developer and Gemini

- **Amazon Q Developer** offers chat, code generation and unit-test generation inside the IDE (and in the AWS console). In this lab it "actually worked" where others struggled, and it has a helpful right-click menu.
- **Gemini Code Assist / Google's assistants** appeared as another extension; Andrew removed Gemini from his sidebar to reduce clutter, a reminder that **too many assistants installed at once compete for your editor**.
- Other names in the roadmap conversation (Chapter 1) include **Claude** (Rola relies on it with Cursor) and other assistants; Andrew's advice is to **try several and keep one or two**.

16.6 Comparison from the lab

Tool	Strength noted	Weakness noted
GitHub Copilot	Deep IDE integration; switchable models; inline chat	Code quality below ChatGPT/Claude in his view; verbose
Windsurf (Cascade)	Best first result with few prompts; builds front end too; runs terminal steps	Credit limits; still hit a dependency error
Cursor	Strong agent mode; easy model switching	Hard-to-find commands; repeated failed fix cost credits
Amazon Q Developer	Worked on the build; good context menu	Less flashy generation

16.7 Practical guidance

KEY IDEA

AI coding assistants speed you up, but you are still the engineer. Read what they generate, run the build, and commit in small steps so you can roll back.

1. **Give context:** open the right folder, mention file names, and paste error messages.
2. **Ask for small, verifiable steps**, not a whole application.
3. **Check dependencies and versions** yourself; generated code often pins the wrong version.
4. **Watch usage limits and credits**; repeated "fix it" loops burn quota.
5. **Keep secrets out of prompts and repos**; use `.env` and `.gitignore` (Chapter 11).
6. **Use version control** so you can diff exactly what the assistant changed.

WATCH OUT

Assistants can **hallucinate APIs** or libraries that do not exist. If a build fails on an import, verify the package name and version before asking the assistant to "fix" it again.

BEYOND THE VIDEO

A useful pattern is to write a short **spec** (README or comments) first, then ask the assistant to implement one function at a time and write **tests** for each. Tests turn its output into something you can trust or reject quickly.

16.8 Self-check

1. Name the difference between an inline-completion extension and an AI-native editor.
2. What are the Cursor shortcuts for edit and chat?
3. Why did repeated "fix it" prompts not solve the Go build error?
4. What are "prompt credits" and why do they matter?
5. Give three habits that keep AI-generated code safe to use.

CHAPTER 17

Building GenAI Apps: UI Generators, Gradio, Streamlit and Backends

Source: video part 2, ~14:00-17:30 (labs on a Vercel v0-style generator, deployment to Vercel, Gradio, Streamlit, a Lovable-style generator, GitHub OAuth and Supabase). Slide OCR shows dashboards; transcript supplies the sequence.

17.1 From model to product

Once you can call a model (Chapter 13) the next question is *how do I put a UI on it?* The labs explore three layers:

Layer	Tools in the labs	Purpose
Prompt-to-UI generators	v0 (Vercel), Lovable, flow/infra generators	Turn a description into a front end (often React/Next.js)
Python UI toolkits	Gradio, Streamlit	Build a chat or image demo around a model in a few lines
Backend and auth	Supabase, GitHub OAuth, SQLite	Store data, sign users in

17.2 Prompt-to-UI generators (v0, Lovable)

These tools **generate a working front end from text** and show a live preview, often with several versions.

- **Infrastructure flow generator:** in one tool you pick what to generate (**GitHub Actions**, **Dockerfile**, **Kubernetes**, **Terraform**) and describe the flow in a prompt; results can be public by default, so avoid sensitive prompts.
- **v0 (Vercel):** produced a **Linktree-style** page with a toggle between preview and code. The data came from a **JSON file** in a **Next.js** route (a **GET** handler), so it looked complete but was **not persisted**; Andrew points this out as a typical gap between a demo and a real backend.
- **Lovable:** generated a whole app in about ten seconds, showing a "version 1", with hooks to **Supabase** for data. It costs around **\$20/month** for serious use, but a minimal free use was enough for the demo.
- **Editors inside generators** are often limiting ("terrible editors, I can't add another row"), so for real work you **download the code** and continue in your own environment (GitPod, Codespaces or local).

Deploying to Vercel

1. Put the generated project in a **GitHub repository** (must be a **personal account** for Vercel's free tier linking).
2. In **Vercel**, create the project and **connect the repo**.
3. Commit and push; Vercel builds automatically. A first error page may be an old cached deployment: check **Deployments** for the build status.
4. Use **--prod** (or promote a deployment) to publish to production.

17.3 Gradio

Gradio (used earlier on SageMaker Studio Lab, Chapter 11) builds ML demos in Python.

```
import gradio as gr

def greet(name):
    return f"Hello {name}!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()          # share=True for a temporary public link
```

Lessons from the session:

- **Interface** is the quick way; **Blocks** gives control over layout. Andrew inspects a **Blocks** demo: markdown at the top, a **Row** containing two text boxes (input and output) and a button, and a **click handler** connecting them. Removing the **Row** broke the layout in his test, so he restored it.
- A **hard-coded value** in an image demo (a fixed probability) showed how easy it is to ship a demo that "does something" without real inference; check that outputs vary with inputs.
- **Streaming** in a chat function needs the function to **yield** partial text; the generated code often **does not match** Gradio's expected format and needs fixes. When the assistant's code failed, pasting the error back usually fixed it, but "these things don't always work as expected".
- Deploying: local `launch()`, a **Space on Hugging Face**, or SageMaker Studio Lab with its own proxy link.

17.4 Streamlit

Streamlit turns a Python script into a web app. Anything placed on its own line (a variable or literal) is **automatically written** to the page ("magic"). Andrew found Streamlit-generated code worked better than most:

- Wrap chat with `st.chat_input` and `st.chat_message`; keep history in `st.session_state`.
- Provide **secrets** via `.streamlit/secrets.toml` (an error such as "secrets not found" means the file or key is missing).
- **Streaming**: `st.write_stream` expects a **generator or stream-like object**; passing a plain string errors ("expect a generator or stream-like object as input"). Using the OpenAI streaming API means reading each chunk from `choices[0].delta.content`, not `message.content`, and the model name you asked for (for example **gpt-4o-mini**) must be set in code: the generator had silently used another model.
- An **image-generating** chat (assistant replies with a DALL-E 3 image) is possible by having the model call the image API and rendering it in an image column.

```
import streamlit as st
from openai import OpenAI

client = OpenAI(api_key=st.secrets["OPENAI_API_KEY"])
st.title("Chat")
if "msgs" not in st.session_state: st.session_state.msgs = []
for m in st.session_state.msgs: st.chat_message(m["role"]).write(m["content"])
if prompt := st.chat_input("Say something"):
    st.session_state.msgs.append({"role": "user", "content": prompt})
    st.chat_message("user").write(prompt)
    stream = client.chat.completions.create(model="gpt-4o-mini",
        messages=st.session_state.msgs, stream=True)
    reply = st.chat_message("assistant").write_stream(
        c.choices[0].delta.content or "" for c in stream)
    st.session_state.msgs.append({"role": "assistant", "content": reply})
```

	Gradio	Streamlit
Sweet spot	Model demos with inputs/outputs; HF Spaces	Data apps and chat UIs
Layout	<code>Interface</code> , <code>Blocks</code> , <code>Row</code>	Top-to-bottom script, <code>session_state</code>
Sharing	<code>share=True</code> , Spaces	Streamlit Community Cloud

17.5 API keys for the app

The lab spends time creating an **OpenAI project and key** (billing and a new project were needed; free tiers are limited). Reminders: put keys in `.env` or `secrets.toml`, never in the repo, and give apps **their own project key** so you can revoke it independently.

17.6 Supabase and login

For persistence Andrew connects the generated app to **Supabase** (a hosted Postgres with auth and an API):

1. Create a **GitHub OAuth app**, with a **callback URL** provided by Supabase; you receive a **client ID** and **client secret** (device flow can stay unchecked).
2. Enable the GitHub provider in Supabase and paste the ID and secret.
3. Create **tables** (for example `links` and `profiles`) and **row-level security policies**, for example "select allowed for all users" so public pages can read data. Without a policy, queries return nothing.
4. Use the project's **URL and anon key** in the app's environment variables.

KEY IDEA

A prompt-to-UI tool gives you a **front end**, not necessarily a **real backend**. Persistence, authentication and access rules (policies) are still your responsibility.

17.7 What to remember

- Generators are great for **prototypes**; download and own the code before production.
- **Gradio** and **Streamlit** are the fastest way to demo a model in Python.
- **Streaming** requires generators and the right response field (`delta`).
- Always separate **secrets** from code, and add **policies** on database tables.

EXAM TIP

You may be asked which tool fits a task: Python ML demo on Spaces = **Gradio**; data app with chat state = **Streamlit**; hosted Postgres + auth = **Supabase**; deploy front end from Git = **Vercel**.

17.8 Self-check

1. Why is a JSON-file demo not a real backend?
2. What must a Streamlit streaming call receive?
3. What is `st.session_state` for?
4. Why do Supabase tables need policies?

5. Where should an OpenAI key live in a Streamlit project?

Hardware, ML Frameworks and Serving Platforms

Source: video parts 2-3, ~16:45-19:55 (labs and slides: FastHTML, Intel OPEA GenAI microservices on EC2, Gaudi, RunPod, Groq, Cloudflare Workers AI; slides on GPUs, CUDA, AWS GPU instances, TensorFlow, Keras). Slide OCR of the hardware slides matches the transcript.

18.1 Why hardware matters

Every earlier chapter had a hidden question: *where does the model run?* Calling an API hides the answer. Once you self-host, you must choose **compute** and a **serving stack**. Andrew's core message in this stretch: compute is expensive, needs are model-specific, and there are **more chip families than just Nvidia**.

18.2 CPU, GPU, TPU and accelerators

Chip	Notes from the course
CPU	General purpose. Small or optimised models can run acceptably on CPUs (llama.cpp, Ollama, Intel Xeon with AMX acceleration). Needs lots of RAM
GPU	Massively parallel; excellent for the matrix maths in neural networks. The dominant choice for training and fast inference
GPGPU	"General-purpose GPU": using a GPU for non-graphics computing (the term behind CUDA)
TPU	Google's Tensor Processing Unit , optimised for TensorFlow and tensors
AI accelerators	Purpose-built chips such as Intel Gaudi (Gaudi 2 and 3), AWS Trainium/Inferentia

CUDA

CUDA is Nvidia's platform that lets developers use CUDA-enabled GPUs for general-purpose computing. It is a software layer, and much of the AI ecosystem (PyTorch, TensorFlow, TensorRT-LLM) is built on it, which is why Nvidia hardware dominates. Alternatives exist (AMD's stack, Intel's oneAPI), but library support varies.

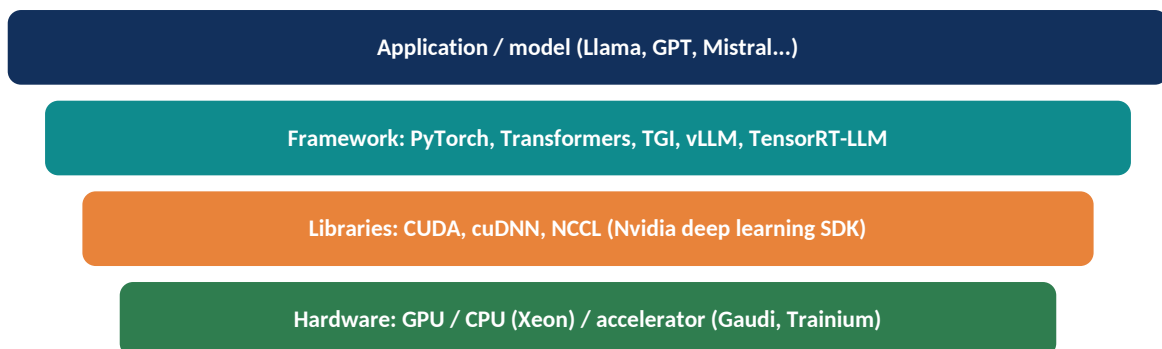


Figure 18.1: The stack from hardware up to the model. Problems can occur at any layer (driver, library, framework version, model).

AWS GPU instance families (as listed, "these could be old")

Instance	GPU
P3	Tesla V100
G3	Tesla M60
G4	T4
P4	A100

AWS keeps changing instance types and newer families carry newer cards (for example H100-class), but the lesson is that **instance family determines the GPU**, and quota approval is often required.

18.3 Frameworks: PyTorch, TensorFlow, Keras

TensorFlow is a low-level deep-learning framework created by the **Google Brain** team, written in Python, C++ and CUDA, with APIs for other languages. Everything is built around the **tensor**, a **multi-dimensional array** (`tf.Tensor`) similar to a NumPy ndarray that can live in **accelerator memory**. Google built the **TPU** specifically for TensorFlow.

Keras is a **high-level abstraction** that runs on TensorFlow (the slide's example model is written in Keras). **PyTorch** (from Meta) is the other dominant framework and the base for Hugging Face **transformers**.

Framework	Level	Typical role
PyTorch	Low-level, flexible	Research and most modern LLM work
TensorFlow	Low-level	Production pipelines, TPUs
Keras	High-level API	Quick model building

18.4 Serving stacks and Intel's OPEA

The section with Intel showcases **OPEA GenAI Components ("GenAIComps")**, "microservices that you can use as the building blocks for your own AI workloads". Each microservice can be configured with different technologies. For example, the **LLM** service sits in front of the thing that actually serves the model, such as **TGI** (Hugging Face **Text Generation Inference**) or **vLLM**. The steps for text generation: open that folder, choose **TGI** as the backend, build the container and run it.

Other pieces in the repository:

- **Data-prep services** with **LangChain** or **LlamaIndex** (folders mirror each other), plus **Ray** to parallelise data preparation.
- **Embedding**, **retriever**, **reranker** and **vector-store** components (the pieces of RAG, Chapter 20).

Launching it on AWS EC2

Andrew launches OPEA on an **Intel Xeon** instance to show GenAI on CPUs:

1. Choose **Ubuntu** and an **m7i** family instance (Sapphire Rapids Xeon); he tried a smaller size, then settled on **4 vCPUs / 8 GB** for the demo.

2. **Storage matters:** the first attempt failed because the disk was too small ("I forgot to launch with enough space"). It is easier to **terminate and relaunch** than to resize.
3. Allow **HTTP/HTTPS** traffic in the security group and attach an **IAM instance profile** for SSM access.
4. Install Docker, clone the repo, build the container images, start the services and test with **curl**.
5. Afterwards **terminate the instance** (and delete any keys you pasted into scripts).

His mantra: "if you do things three times, then you absolutely know how to do it".

18.5 Gaudi, Tiber Developer Cloud and RunPod

- **Intel Gaudi** accelerators: Gaudi 3 is the latest; access on the cloud required **account approval**, while Gaudi 2 was available through IBM. The Intel developer cloud offers **learning notebooks** that show how to **fine-tune Llama with Hugging Face** and run inference.
- **RunPod** rents GPUs by the hour with prebuilt Jupyter templates (Andrew could not purchase credits because of card issues, so he only described the flow: pick a template, launch, open the notebook).

18.6 Groq

Groq advertises very fast inference on its own **LPU** chips. Points shown:

- A chat playground with **voice input** using **Whisper large v3**; responses appeared almost instantly, "they are trying to emphasise how fast their inference is".
- Very low per-token prices and a **free tier with low rate limits**; pay-per-token for individuals was "coming soon" at the time, with company billing first.
- Groq is also the backend he used for the **Instructor** structured-output demo (Chapter 15).

18.7 Cloudflare Workers AI

Workers AI lets you call open models (text, image, speech) from **serverless Workers** on Cloudflare's network.

- Start with **npm create cloudflare@latest** (the **create-cloudflare** package, "C3") to scaffold a **Worker** or Pages project.
- The image example ("a photo of a short-haired dachshund") used a Stable-Diffusion-class model and returned an image through a small route; default parameters were used.
- Errors like "response is not defined" came from scoping in the generated code; he trimmed the inputs (chat, response) to one to debug it.
- Local development (**wrangler dev**) reloads a local server; **remote AI bindings** still call Cloudflare's GPUs.

18.8 Choosing where to run

Need	Option
No infrastructure	Provider API or Bedrock/Vertex/Azure (Chapter 13)
Fastest hosted inference	Groq
Serverless edge functions	Cloudflare Workers AI
Full control, GPU by the hour	Lightning AI, RunPod, cloud GPU instances
CPU-only, private, cheap	Ollama / llama.cpp / OPEA on Xeon
Fine-tuning at scale	Multi-GPU cloud instances (A100/H100) or Gaudi

KEY IDEA

Serving = model weights + an inference engine (TGI, vLLM, TensorRT-LLM, llama.cpp) + hardware + an API layer. Managed services bundle all four; self-hosting means you assemble them.

EXAM TIP

Match the term: **CUDA** = Nvidia GPU compute platform; **TPU** = Google tensor chip; **TGI** = Hugging Face serving server; **Gaudi** = Intel AI accelerator; **Keras** = high-level API on TensorFlow.

18.9 Self-check

1. What is CUDA and why does it matter to the AI ecosystem?
2. Name two model-serving engines mentioned.
3. What went wrong in the first EC2 launch and how was it fixed?
4. Why is Groq highlighted?
5. What is a tensor?

Model Size, Memory and Optimisation

Source: video part 3, ~19:50-20:30 (slides "Flash Attention", "Quantization", "Knowledge Distillation", "GenAI size comparisons" presented with Rola Dali, plus the Chinchilla graph on parameter growth).

19.1 What does "size" mean?

Rola Dali's slides open with the key definition: ML models are often **sized by their number of parameters**, and the parameters **are the model weights**. Abstractly, a neural network is a **complex mathematical model**. The same architecture trained on two different data sets ends up with **two different sets of parameters**.

Fact from the session	Value
Range of model sizes	From a handful of parameters up to about 2 trillion (GPT-class models are around 1.8 trillion)
Growth trend	Model sizes have grown roughly 10x per year in the graph; the Transformer started around 50 million parameters
How to run a 2T model	It must be distributed : shard the model and distribute the computation across many devices
Common local sizes	7B to 8B; Qwen around 32B; 70B needs serious hardware

Bigger models can encode more, "like a bigger brain (not literally true in neuroscience, but in an abstract way)". But the **Chinchilla lesson** applies (Chapter 10): a huge model that does not see enough **good data** does not learn as much. Model, **data quality/quantity** and **hardware/infrastructure** are three linked constraints.

19.2 Memory arithmetic

The most useful skill in this chapter is estimating memory from parameter count.

Precision	Bytes per parameter	1 billion parameters	7 billion	70 billion
FP32 (32-bit float)	4	4 GB	28 GB	280 GB
FP16 / BF16	2	2 GB	14 GB	140 GB
INT8	1	1 GB	7 GB	70 GB
INT4	0.5	0.5 GB	3.5 GB	35 GB

(Weights only; you also need memory for the **KV cache**, activations and the runtime.)

The reasoning in the video: a byte is 8 bits, a 32-bit float is 4 bytes, so **1 billion parameters at FP32 need 4 GB of RAM** just for the parameters. Many providers use **16 bits (2 bytes)** instead.

Training needs far more

Rola explains why training needs more memory than inference: the model starts with parameters **initialised at random**, makes a prediction, calculates how far off it is, and adjusts every parameter. Extra **metadata** (gradients and optimiser state) must be stored for each parameter. Her worked example: **full fine-tuning of a 1B-parameter model takes about 80 GB**, roughly **20x** the 4 GB of the weights, and would fit on a high-end GPU such as an A100 (an H100 is newer and stronger).

KEY IDEA

Rule of thumb: **inference $\approx$ parameters $\times$ bytes per parameter (+ overhead)**; **full training $\approx$ many times more** because of gradients and optimiser state. This is why **PEFT/LoRA** (Chapter 8) is popular.

19.3 Number formats

Rola walks through how computers encode numbers, because precision choices are the heart of quantization:

- **8 bits = 1 byte**. As an *unsigned integer*, 8 bits encode **0 to 255**; if signed, the first bit encodes the sign.
- A floating-point number splits its bits into **sign, exponent and fraction (mantissa)**. Using an **exponent** gives a far larger range.
- **FP16** uses 16 bits with a limited range (the transcript quotes about $\pm 65,504$), while **BF16 (bfloat16)** keeps the *same 16 bits* but **allocates more bits to the exponent and fewer to the fraction**, so it covers a **much wider range** with less precision, which suits deep learning.
- Which numbers you choose for the exponent/fraction split is a design decision made by whoever sets up the system; **you should normally use the precision recommended** for the model.

19.4 Quantization

Quantization is the process of reducing the precision of a signal from higher to lower precision. Applied to LLMs: store weights (and sometimes activations) in fewer bits.

Andrew's picture: a smooth blue wave (high precision) becomes a blocky staircase after quantization, but **the overall shape stays the same, so it should behave similarly**. He also uses the JPEG analogy: lower quality means a smaller, faster-loading file but less accurate.

Benefit	Cost
Smaller memory footprint	Potential loss of quality
Cheaper hardware; larger models fit	Techniques vary and can be complex
Sometimes faster inference	Not always faster

Insight from the session: when Andrew tried quantizing, the files did not always shrink and inference was not always faster. Rola's answer: the real gain is often **memory saved**, which lets you run a bigger model or more concurrent requests; speed gains depend on **hardware support** for the lower-precision format.

Examples named: **QLoRA** (quantized LoRA fine-tuning), **GGUF** files, and quantization-aware libraries. Implementation is math-heavy; in practice you **look up the recommended settings** and use them.

WATCH OUT

More extreme quantization can noticeably hurt quality for small models. Test on *your* task after quantizing, not just on a general benchmark.

19.5 Knowledge distillation

Knowledge distillation transfers knowledge from a **large teacher model** to a **smaller student model** so the student performs the same task **faster and at lower resource cost**. The goal is generally a **small language model**. Simplest explanation: the teacher's predictions (**soft targets**) and the ground-truth labels (**hard targets**) are both used to train the student to behave like the teacher.

Models with "distilled" in the name (for example distil-whisper) went through this process (Chapter 5). The slide's performance example is **Nvidia Nemotron → Minitron**, produced with **pruning plus distillation**, giving smaller versions that keep most of the quality.

19.6 Flash attention

Flash Attention is a **memory-efficient, faster variant of the attention mechanism**, optimised for **GPUs**, to handle **longer sequences with less computation and memory**. Andrew's advice: "the only thing to remember is that it is a better version of the traditional attention mechanism."

- It comes in **versions 1, 2 and 3**; **3 is not necessarily better than 2**, it depends on the hardware: **Hopper GPUs (H100)** benefit from Flash Attention 3.
- Efficiency comes from computing attention scores in **small chunks** and **fusing operations** such as softmax and matrix multiplication to minimise memory traffic.

19.7 Fine-tuning example on Lightning AI

The slide shows a **LoRA fine-tune of a small Llama (1B-3B class)**: provide a **JSON data set**, set **epochs**, **output directory** and **precision**, run the trainer, and serve the result with **LitGPT**. The hardware used: an **Nvidia L4 GPU** (24 GB) with 16 vCPUs and 64 GB RAM. It shows that small-model fine-tuning fits on modest cloud GPUs.

19.8 Optimisation toolkit

Technique	What changes	Effect
Quantization	Number precision	Less memory; maybe faster
Pruning (Chapter 8)	Remove parameters	Smaller, faster; some loss
Distillation	Train a small student	Small model, similar behaviour
Flash attention	Attention algorithm	Longer sequences, less memory
Prompt caching (Chapter 15)	Reuse computed context	Lower latency and cost
Sharding / distributed inference	Split model across devices	Runs models too big for one GPU
PEFT / LoRA	Train fewer parameters	Cheap fine-tuning

EXAM TIP

Given a model size and precision you may need to estimate memory: **parameters × bytes per parameter**. Know that **BF16 has the same size as FP16 but a wider range**, and that **quantization trades precision for footprint**.

19.9 Self-check

1. How much memory do 7 billion FP16 parameters need for weights alone?
2. Why does full fine-tuning need so much more memory than inference?
3. Difference between soft targets and hard targets in distillation?
4. What makes BF16 different from FP16?
5. When would Flash Attention 3 be preferred?

RAG and Vector Databases

Source: video parts 3-4, ~20:25-22:20 (labs on Pinecone, Elasticsearch, Amazon OpenSearch and Bedrock knowledge-base pricing, Postgres vector store, sentence-transformers, web scraping). OCR of the dashboards confirms the products; the transcript supplies the steps.

20.1 Retrieval-augmented generation

RAG gives an LLM information it was not trained on by **retrieving relevant documents at question time** and placing them in the prompt. Earlier chapters set the scene: knowledge mining (Chapter 9) has the same ingest, enrich, explore shape; embeddings (Chapter 7) make similarity search possible; and the context window (Chapter 7) limits how much you can paste in.

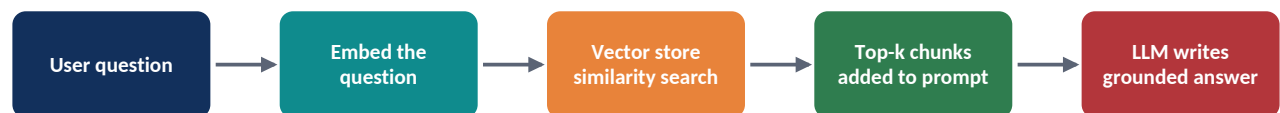


Figure 20.1: The RAG loop. Documents are embedded once and stored; each question is embedded, matched, and the best chunks are added to the prompt.

Two phases

Phase	Steps
Indexing (offline)	Load documents, clean , split into chunks , embed each chunk, upsert vectors and metadata into a vector store
Query (online)	Embed the question, similarity search for the top-k chunks, build a prompt (question + retrieved text), call the LLM, return the grounded answer

KEY IDEA

RAG changes **what the model sees**, not its weights. It is the first tool to reach for when you need **private, fresh or citable knowledge** (Chapter 8's comparison table).

20.2 Vector databases and stores

A **vector store** keeps embeddings and answers "which vectors are nearest to this one?" quickly using an **approximate nearest-neighbour (ANN)** index. Metadata (original text, source, tags) is stored beside each vector so results can be shown and filtered.

Product	Type	Notes from the labs
Pinecone	Managed vector database	Simple Python SDK; create an index, upsert vectors, query
Elasticsearch / Elastic Cloud	Search engine with vector and full-text search	Hybrid search (keywords + vectors); API keys with scoped access

Product	Type	Notes from the labs
Amazon OpenSearch	AWS managed search/vector engine	Used with Bedrock knowledge bases
PostgreSQL + pgvector	Relational DB with a vector extension	Use the database you already have
Others named in the roadmap	Chroma, Weaviate, Milvus, FAISS (library)	Not all demoed

20.3 Lab: Pinecone

Steps and lessons from the session:

1. Create an account and a **Pinecone API key**; install with `pip install pinecone`.
2. **Initialise the client** with the key (an SDK change meant an older "OpenAI config" argument was **no longer supported**; the current getting-started guide shows the correct call. **SDKs change, so follow the latest docs**).
3. Create an **index** with the embedding **dimension** that your model outputs (the index dimension must match the embedding size).
4. **Upsert** records (id, vector, metadata). The response reports the upserted count (6 in the demo). The index may not show up in the dashboard **until data has been loaded**, and the dashboard can lag; delete duplicate API keys created by retries.
5. **Query**: embed a question such as "What caused the 1929 Great Depression?" and retrieve the closest passages. The result includes the **original text stored in metadata**, and rewording the question slightly ("recession" instead of "depression") still finds the same passage, which is the point of **semantic search**.

```
from pinecone import Pinecone
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index("demo")
index.upsert(vectors=[("id1", embedding, {"text": chunk})])
res = index.query(vector=q_embedding, top_k=3, include_metadata=True)
```

20.4 Lab: Elasticsearch

- Create a deployment on **Elastic Cloud** and an **API key** with the right scope ("full read access" everywhere is convenient for a lab but too broad for production; note the interface offers per-index/cluster scoping).
- Two auth styles appear: an **API key** (encoded) or **ID plus key**; a mismatch with the docs produced errors until he created a new key.
- Andrew found **out-of-date sample code** (it used GPT-3.5-turbo, was not written for a notebook, and a search query variable was never set). He fixed it by making the **multi-match query** use the user's question. Lesson: **vendor examples rot; read and adapt them**.
- Elastic supports **hybrid retrieval**: BM25 keyword scoring and vector similarity in the same query.

20.5 Lab: Amazon OpenSearch and Bedrock knowledge bases

- A dataset is loaded with **pandas**; a **"combined attributes" column** is created that merges several fields into one text field for full-text search and for embedding (a common technique).
- **Embeddings** are generated (with Bedrock or a sentence model), documents are indexed in an **OpenSearch domain/cluster**, and the master user and password are set (special characters get URL-encoded).

- Bedrock **knowledge bases** wrap this pattern: it manages chunking, embedding and retrieval for you. Andrew investigated **cost**: a knowledge base is not free just by existing; the vector store behind it (often OpenSearch Serverless) has a **minimum running cost**, and queries may be billed too (he recalled pricing per thousand queries). **Always check the pricing page and delete clusters when done**; he deletes his cluster immediately to avoid forgetting.

20.6 Lab: Postgres as a vector store

- Connect to Postgres, enable the **pgvector** extension (the notebook noticed the extension and set up the table automatically).
- The sample used **fake embeddings** ("replace with a real query embedding"), so he swapped in a **real open-source embedding model: all-MiniLM-L6-v2 from Sentence Transformers** (Chapter 5).
- Then run a **similarity search** with a SQL operator for vector distance.

```
-- pgvector examples
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE docs (id serial, body text, embedding vector(384));
SELECT body FROM docs ORDER BY embedding <=> $1 LIMIT 3;  -- cosine distance
```

20.7 Adding web data: scraping and search

A RAG app often needs fresh web content. Andrew asks an assistant for a way to scrape a page and strip HTML: **Requests** plus **Beautiful Soup 4** ("the Nokogiri of Python"). He also tries a **search API** (SerpAPI) to fetch results; the import name differs from the pip package name, which caused an import error until he checked the docs. He then embeds the cleaned text and stores it, forming the "ingest" leg of RAG.

20.8 Chunking, embedding and retrieval choices

BEYOND THE VIDEO

These decisions affect quality more than the choice of vector database.

Decision	Guidance
Chunk size	Typically a few hundred tokens; small enough to be specific, big enough to keep context. Add overlap so ideas are not cut in half
Embedding model	Use the same model for documents and queries; check dimension and language coverage
top-k	Retrieve a handful (3-8) and keep total tokens inside the context window
Hybrid search	Combine keyword and vector search for names, codes and rare terms
Reranking	A reranker (for example Cohere Rerank) reorders results for relevance
Metadata filters	Restrict by source, date or permissions
Evaluation	Test with known questions; check whether the right chunk was retrieved before blaming the LLM

WATCH OUT

If retrieval brings back the wrong text, the LLM will confidently answer from it. **Debug retrieval first.** Also prevent **prompt injection** hidden in retrieved documents, and respect **access control** so users only retrieve what they may see.

20.9 RAG vs fine-tuning (recap)

	RAG	Fine-tuning
Updates	Change documents instantly	Retrain to update
Cost	Vector store + embeddings	Training compute
Traceability	Can cite sources	Knowledge is baked in
Best for	Facts, policies, documentation	Style, format, task behaviour

EXAM TIP

RAG components: **embedding model, vector store, retriever, (reranker), LLM**. A question about "adding company documents without retraining" points to **RAG**.

20.10 Self-check

1. Why must the index dimension match the embedding model?
2. What extra column technique helps full-text and vector search?
3. Why did Andrew delete the OpenSearch cluster right away?
4. What is hybrid search?
5. If answers are wrong, what should you check first?

Agents: OpenHands and CrewAI

Source: video part 4, ~22:15-22:53 (labs: OpenHands (formerly OpenDevin) in Docker, a CrewAI "game builder" crew, and a Pygame reinforcement-learning demo). This is the last section of the course.

21.1 What is an agent?

In Chapter 12, **ReAct** showed the core loop: *thought, action, observation, repeat*. An **agent** is a program that wraps an LLM in that loop and gives it **tools** (run code, browse, edit files, call APIs) so it can pursue a goal over several steps instead of answering once. Agents differ from assistants (Chapter 10) and code editors (Chapter 16) in one way: **they act, observe results, and decide the next step by themselves**.

Concept	Meaning
Tool	A function the agent may call (shell, web search, file editor, API)
Memory / state	What it remembers across steps (conversation, files, notes)
Planner / loop	Decides the next action from observations
Multi-agent crew	Several agents with roles that hand work to each other

Andrew's view from Chapter 12 still applies: frameworks automate the loop, but it is better to **know how the manual pieces work**.

21.2 OpenHands (formerly OpenDevin)

OpenHands began as **OpenDevin**, an open-source alternative to **Devin**, the "agentic coding agent that writes code for you". It was renamed **OpenHands**. Key points from the lab:

- It **runs in Docker**; Andrew used his **Intel machine** for the run because Docker was already configured (and says to use your local machine if you can run containers).
- You choose a **model and provide an API key** (a paid model gave better results; free tiers may fail). Costs are yours.
- The agent can **write code and execute it**: asked to make and run a `hello.sh` file, it wrote the script **and ran it**, which Andrew found "really interesting".
- It can also create **Jupyter notebooks**: "make a simple notebook to use BERT for classification", and explained what it was doing step by step, asking you to continue.
- Earlier experience: a previous attempt for the video had been complicated; this one worked better.

WATCH OUT

Autonomous agents can run commands on your machine. Run them in **containers or sandboxes**, review actions, and never give them credentials you would not want to leak.

21.3 CrewAI

CrewAI is a Python framework for **multi-agent "crews"**: you define **agents** (role, goal, backstory), **tasks**, and how they collaborate. The lab example is a **game-builder crew** that writes a small game.

Setup lessons:

1. The project uses **Poetry** (a Python dependency and packaging tool, like Ruby's Bundler). Andrew had not installed it, so: `pip install poetry`, then `poetry install`. **Commands are then run inside Poetry's environment** (`poetry run <script>`), just as Ruby users run `bundle exec`.
2. You can edit the **agents** and **tasks** files to change behaviour, and configure which model to use ("you may incur cost; I'm using a paid version; on the free version this might fail").
3. Run the crew with the project's script, for example `poetry run game_builder_crew`.
4. The generated game uses **Pygame**, which needs a windowing environment. Andrew installed it on the **Windows side**, not inside WSL, since it needs the Windows GUI layer (it should work on Mac; Linux he was unsure).

21.4 A reinforcement-learning demo

The final example ties back to Chapter 3. The Pygame project had **two entry points**, `train` and `run`. Training lets an agent play **many iterations**, receiving rewards, until it gets good; then `run` uses the learned policy. Andrew identifies it as **reinforcement learning** (the Mario/Sonic style of learning from trial and error) and shows the agent playing the game. He ends by saying this is "super powerful and really awesome", and wonders why he does not use it more.

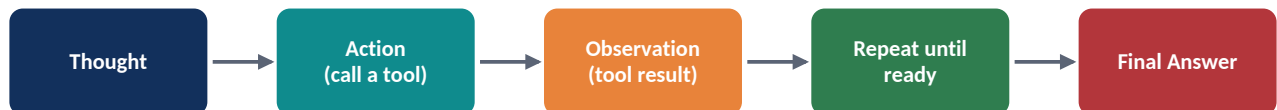


Figure 21.1: The loop shared by ReAct, coding agents and crews. What changes is the number of tools and agents.

21.5 Design tips for agents

BEYOND THE VIDEO

General practice, beyond the demos.

Tip	Why
Start with a single agent and two or three tools	Easier to debug than a crew
Write clear tool descriptions	The model chooses tools from the text you give it (Chapter 12: it invented tools it was not told about)
Set step and cost limits	Prevent endless loops and surprise bills
Log every thought, action and observation	Debugging and audit

Tip	Why
Add human approval for risky actions	Deleting files, spending money, sending email
Prefer structured outputs (Chapter 15) for tool arguments	Fewer parsing errors

21.6 The end of the course

Andrew closes by saying he does not know why he does not use these tools more, because he knows about them, and signs off. The full journey of the course, as this guide has followed it, is: **foundations, then models, then prompting, then APIs, then local and cloud serving, then apps, then optimisation, then RAG and agents.**

EXAM TIP

Agent = LLM + tools + a loop. OpenHands (coding agent, Docker) and CrewAI (multi-agent crews) are named examples. Reinforcement learning = learning by trial, error and reward.

21.7 Self-check

1. What extra capability do agents have compared with chat assistants?
2. Why should agents run in a container or sandbox?
3. What is Poetry used for in the CrewAI lab?
4. Why did Pygame need to be installed outside WSL?
5. Which learning type does the train/run game demo illustrate?

Study Aids: Cheat Sheet, Glossary and Practice Questions

Built from the whole course (video parts 0-4, transcript and slide OCR). Use this section for revision after reading the chapters.

S.1 One-page cheat sheet

Topic	Remember
AI nesting	AI > ML > deep learning > generative AI; LLMs are foundation models that use the Transformer
ML types	Supervised (labeled), unsupervised (unlabeled), reinforcement (reward); tasks: regression, classification, clustering
Neural nets	Weights = parameters; back-propagation adjusts them; ReLU family for hidden layers, sigmoid/tanh for bounded output, softmax for multi-class
Encoder vs decoder	Encoder-only (BERT) understands; decoder (GPT) generates; encoder-decoder (T5) transforms
Tokens	Text becomes token IDs; context window = input + output tokens; BPE, WordPiece, SentencePiece
Embeddings	Vectors where similar meaning is near; power semantic search and RAG
Parameters vs hyperparameters	Learned weights vs set-by-you (learning rate, epochs, batch size)
Datasets	Train (learn), validation (tune), test (final unbiased check); ground truth = correct labels
Fine-tuning	New weights, not new code; full vs PEFT/LoRA vs last-layer; instruction vs domain-specific
Shrinking models	Pruning, quantization (lower precision), distillation (teacher to student)
Prompting	Zero-shot, few-shot, chaining, CoT, self-consistency, ToT, CO-STAR, ReAct
Serving	API (Bedrock/Vertex/Azure/OpenAI) vs local (Ollama, llama.cpp, llamafile, GGUF) vs cloud GPUs
Structured output	Grammar/FSM/regex constraints, Pydantic, Instructor
RAG	Chunk, embed, store, retrieve, augment prompt, generate; debug retrieval first
Memory maths	Weights GB ≈ billions of params × bytes per param (FP32=4, FP16/BF16=2, INT8=1, INT4=0.5)
Responsible AI	Fairness, explainability, privacy/security, safety, controllability, veracity/robustness, governance, transparency
Exam	65 questions, ~75% (750) to pass, 2 h exam / 160 min seat time, no expiry

S.2 Glossary

Term	Definition
Activation function	Gate in a node that turns summed inputs into an output (ReLU, sigmoid, tanh, softmax)
Agent	LLM plus tools in a reason-act-observe loop
Attention	Weights that say how much each token matters to the others
Back-propagation	Adjusting weights backwards from the error
Base / foundation / pre-trained model	The general model before task-specific tuning
Benchmark	Standard test set with known answers (MMLU, HumanEval, LiveBench)
BERT	Encoder-only transformer from Google (2018)
BF16	16-bit float with a wide exponent range
Chain of thought	Prompt style asking for step-by-step reasoning
Classification	Predicting a category from labeled data
Clustering	Grouping unlabeled data by similarity
Context window	Maximum tokens (input + output) the model can handle
Corpus	Large structured collection of text
CUDA	Nvidia's GPU computing platform
Data wrangling	Cleaning and transforming raw data
Distillation	Training a small model to mimic a large one
Embedding	Vector representation of data
Epoch	One pass through the training data
Feature	Numeric characteristic used as model input
Few-shot	Prompt containing examples (in-context learning)
Fine-tuning	Retraining pre-trained weights on a smaller data set
Flash attention	Memory-efficient GPU attention algorithm
GGUF	Single-file quantized model format for llama.cpp/Ollama
Ground truth	Properly labeled data used as the standard
Hallucination	Fluent but false output
Hyperparameter	Setting chosen before training
Inference	Requesting and receiving a prediction
LoRA / PEFT	Parameter-efficient fine-tuning updating few weights
LLM	Foundation model using the Transformer for language
Loss function	Measure of prediction error
Model card	Documentation page for a model

Term	Definition
Multi-head attention	Several attention heads in parallel
NER	Named-entity recognition
Open weight	Weights downloadable, data/code not necessarily public
Parameter	Learned weight inside the model
Positional encoding	Adds word-order information to embeddings
Prompt caching	Reusing computed context to save time and cost
Pruning	Removing parameters to slim a model
Quantization	Storing numbers at lower precision
RAG	Retrieval-augmented generation
ReAct	Thought, action, observation loop
Regression	Predicting a continuous value
Reinforcement learning	Learning by trial and reward
Self-attention	Attention within one sequence
SFT	Supervised fine-tuning on labeled data
Tokenization	Splitting text into tokens with IDs
Transformer	Architecture built on attention (Google, 2017)
Vector database	Store optimised for nearest-neighbour search

S.3 Practice questions

Try each before reading the answer below.

1. A model must predict next month's sales figure from past data. Which task is this?
2. You have unlabeled customer records and want natural groups. Which task?
3. Which activation function outputs probabilities that sum to 1 across classes?
4. What fixes the "dying ReLU" problem?
5. Which transformer part does BERT use?
6. What are BERT's two pre-training tasks?
7. Which set gives the final unbiased evaluation of a model?
8. Learning rate is a parameter or a hyperparameter?
9. What does the context window include?
10. Which technique passes one prompt's output into the next?
11. What is in-context learning?
12. Which technique trains only a small subset of parameters?
13. Difference between quantization and distillation?
14. What memory do 13 billion FP16 parameters need for weights?

15. Which file format do llama.cpp and Ollama use?
16. What is the main risk of "please output JSON" without constraints?
17. Which component of RAG turns text into vectors?
18. If a RAG system answers wrongly, what do you inspect first?
19. What does an agent add to an LLM?
20. Name two things you should delete after a cloud lab to avoid charges.

S.4 Answers

1. **Regression.**
2. **Clustering** (unsupervised).
3. **Softmax.**
4. **Leaky ReLU, parametric ReLU or ELU.**
5. **The encoder** (encoder-only).
6. **Masked language modelling and next-sentence prediction.**
7. **The test set.**
8. **Hyperparameter** (set before training).
9. **Input plus output tokens** combined.
10. **Prompt chaining.**
11. Giving examples inside the prompt so the model adapts without changing weights.
12. **PEFT (for example LoRA).**
13. Quantization lowers **numeric precision** of the same model; distillation trains a **smaller student** to imitate a teacher.
14. $13 \times 2 =$ **about 26 GB** (plus overhead).
15. **GGUF.**
16. Invalid or inconsistent JSON; use grammar/regex constraints or a library such as Instructor.
17. The **embedding model**.
18. **Retrieval**: were the right chunks returned?
19. **Tools and a decision loop**, so it can act and observe.
20. **Deployments/endpoints, runtimes, instances, clusters and notebooks** (and revoke exposed keys).

S.5 Course map and where each topic lives

Chapter	Content	Video position
1	Orientation, exam, roadmap	Part 0, 0:00-0:53
2-4	AI/ML basics, neural networks	Part 0, 0:54-1:27 and 2:38-2:53
5	BERT and sentence transformers	Part 0, 1:27-2:38
6-7	Terminology, responsible AI, transformers	Part 0, 2:53-3:25
8-9	Fine-tuning and data topics	Part 0, 3:24-3:48
10	Choosing models and assistants	Part 0, 3:48-4:25
11	Development environments	Part 0-1, 4:25-6:10
12	Prompt engineering	Part 1, 6:05-7:00
13-14	Model APIs, Hugging Face	Part 1, 6:55-10:10
15	Local models, structured output	Part 1-2, 10:10-11:50
16-17	Coding assistants, app building	Part 2, 11:50-17:30
18-19	Hardware, serving, size and optimisation	Parts 2-3, 16:45-20:30
20-21	RAG and vector stores, agents	Parts 3-4, 20:25-22:53

S.6 How to study this guide

1. Read chapters 2 to 9 first; they carry most of the **exam** vocabulary.
2. Do the self-check at the end of each chapter without looking.
3. Re-do the practice questions after a day.
4. Try one lab per stage (an API call, a local model with Ollama, a small RAG) to make the ideas concrete.
5. Take the free practice exam, then review anything under 75%.